

Exqutor 관련 References 81편 통합 요약

범위: References [1]~[81] 중 81편의 핵심 내용 요약

[1] AnalyticDB-V; A Hybrid Analytical Engine Towards Query Fusion for Structured and Unstructured Data 총정리

역할군: (A) VAQ 개념 원조: 본 논문 문제 정의의 출발점

AnalyticDB-V는 Exqutor가 풀려는 문제, 즉 "벡터 검색과 SQL 분석 쿼리를 하나의 시스템 안에서 효율적으로 처리하는 것"을 최초로 산업 규모에서 구현한 시스템이다. 현대 데이터 파이프라인에서 구조화 데이터(테이블)와 비구조화 데이터(이미지, 텍스트 등)가 함께 존재하는데, 기존에는 이 두 종류를 별개의 시스템으로 처리해야 했다. AnalyticDB-V의 핵심 제안은 **벡터 유사도 검색을 SQL의 물리적 연산자로 구현**하는 것으로, 사용자가 하나의 SQL 문으로 벡터 검색과 관계형 분석을 동시에 표현할 수 있도록 한다.

시스템은 Lambda 아키텍처를 채택하여 스트리밍 레이어와 배치 레이어를 분리하고, VGPQ(Vector Graph Product Quantization)라는 새로운 ANN 검색 알고리즘을 제안한다. 가장 중요한 기여는 **정확도 인식 비용 기반 최적화(Accuracy-Aware Cost-Based Optimization)**로, ANN 검색의 본질적 근사성을 옵티마이저에 통합하여 정확도 요구사항과 비용 간의 트레이드오프를 관리한다. 다만 **카디널리티 추정** 문제를 깊이 다루지 않았으며, 이것이 Exqutor가 보완하는 핵심 빈틈이다. 본 논문과의 관계: VAQ(Vector-Augmented Analytical Query) 개념의 원형을 제시했으나, "벡터 검색 결과가 정확히 몇 건이나 나올지 추정하는 문제"는 미해결로 남겨두어, Exqutor의 근본적 동기를 제공한다.

현대 데이터 파이프라인에서는 **구조화 데이터**(매출, 재고, 사용자 정보 등 숫자/텍스트 테이블)와 **비구조화 데이터**(이미지, 동영상, 오디오, 텍스트 문서 등)가 함께 생성된다.

기존에는 이 두 종류의 데이터를 별개의 시스템으로 처리했다:

- 구조화 데이터 → 관계형 데이터베이스 (MySQL, PostgreSQL 등)
- 비구조화 데이터 → 별도의 벡터 검색 엔진 (Faiss 등)

문제는 실제 비즈니스에서 두 데이터를 **함께** 분석해야 하는 경우가 매우 많다는 것이다. 예를 들어:

- "이 상품 이미지와 비슷한 상품을 찾되, 가격이 100만원 이하이고 재고가 있는 것만 보여줘"
- "이 얼굴 사진과 유사한 사용자를 찾아서, 그 사용자의 최근 구매 이력을 분석해줘"

이런 쿼리를 처리하려면 개발자가 두 시스템 사이에서 수동으로 데이터를 옮기고 결합해야 했다. 이 과정이 매우 복잡하고 느렸으며, 최적화 기회를 완전히 놓치게 된다.

AnalyticDB-V는 **Lambda 아키텍처**를 채택한다:

스트리밍 레이어(Streaming Layer): 실시간으로 들어오는 벡터 데이터를 처리하며, 새로 삽입된 벡터는 즉시 검색 가능하다.

배치 레이어(Batching Layer): 주기적으로 새로 삽입된 벡터들을 압축하고, ANNS 인덱스를 재구축한다.

[2] Hybrid RAG-Empowered Multi-Modal LLM for Secure Data Management in Internet of Medical Things 총정리

핵심 문제는 의료 IoT 환경에서 X-ray 이미지, 심전도 시계열, 텍스트 진료 기록 같은 멀티모달 데이터를 통합 분석해야 한다는 것이다. 논문은 세 가지 계층으로 구성된 시스템을 제안한다: (1) 멀티모달 하이브리드 RAG로 각 모달리티별 후보를 검색하고 교차 검증, (2) 블록체인 기반 안전한 데이터 공유, (3) AoI(Age of Information) 메트릭으로 데이터 신선도를 보장하는 인센티브 메커니즘. 특히 첫 번째 계층의 RAG 파이프라인이 바로 **복합 VAQ의 실체**이며, 정확한 카디널리티 추정이 없을 때 얼마나 비효율적인지를 보여준다.

본 논문과의 관계: VAQ의 실제 구현에서 벡터 유사도 검색과 메타데이터 필터링, 다중 모달리티 교차 검증이 결합될 때, 정확한 카디널리티 추정이 얼마나 중요한지를 실제 의료 응용에서 입증한다.

IoMT(Internet of Medical Things) 환경에서는 병원·웨어러블 기기·원격 진료 시스템이 대량의 **멀티모달 의료 데이터**를 생성한다:

- X-ray, CT, MRI 같은 **이미지 데이터** (2D/3D)
- 심전도(ECG), 맥박 센서 같은 **시계열 데이터** (시간 의존적)
- 의사 노트, 처방전, 임상 검사 결과 같은 **텍스트 데이터** (비정형)
- 나이, 성별, 기저질환 같은 **구조화된 메타데이터** (정형)

이 데이터를 기반으로 LLM이 정확한 의료 보조 판단을 내리려면, **검색 증강 생성(RAG)** 파이프라인이 필수적이다. 하지만 기존 RAG 시스템에는 세 가지 핵심 문제가 있다:

문제 1 — 데이터 신선도(Freshness):

의료 데이터는 실시간 업데이트되지만, RAG에 제공되는 검색 결과가 오래된 데이터일 경우 LLM의 판단이 부정확해진다.

예를 들어:

- 환자의 가장 최근 혈액 검사 결과(오늘)가 아닌 3개월 전 결과를 참조하면
- 잘못된 진단으로 이어질 수 있다 (예; 수치가 개선되었는데 악화된 것으로 판단)

문제 2 — 보안과 신뢰:

의료 데이터는 극도로 민감하다. HIPAA, GDPR 같은 규제를 만족하면서도:

- 여러 병원이 데이터를 공유할 때, 중앙 기관 없이 어떻게 신뢰를 확보할 것인가?

[3] Tree-Based RAG-Agent Recommendation System; A Case Study in Medical Test Data 총정리

의료 검사 추천 시스템의 핵심은 "환자의 증상을 보고, 어떤 진료과에 보낼지, 그리고 어떤 검사를 처방할 것인가"라는 결정이다. 이것은 단순하지 않은 **계층적 추론**을 요구한다. HiRMed (Hierarchical RAG-enhanced Medical Test Recommendation) 시스템은 트리 구조로 이를 구현하는데, 각 노드는 전문화된 RAG 에이전트이고, 상위 계층부터 하위 계층으로 점진적으로 검색 공간을 좁혀나간다.

이 구조를 SQL VAQ로 표현하면, 각 트리 레벨에서 카디널리티가 크게 달라지는 **다단계 조인과 벡터 범위 검색의 결합**이 된다. 정확한 카디널리티 추정이 없으면, 상위 레벨의 부정확한 카디널리티가 하위 레벨로 전파되어 최악의 경우 지수적 성능 저하를 초래한다.

본 논문과의 관계: 의료 도메인에서 트리 구조의 각 노드별로 서로 다른 카디널리티 특성을 갖는 VAQ가 발생하며, 이것이 정확한 카디널리티 추정의 필요성을 강하게 보여준다.

의료 검사 추천 시스템에서 "환자의 증상을 보고, 어떤 진료과에 보내고, 어떤 검사를 처방할 것인가"라는 결정은 결코 단순하지 않은 **계층적 추론**을 요구한다.

기존 LLM의 한계:

일반 목적(General-Purpose) LLM에 "환자가 심한 두통을 호소한다"고 입력하면:

GPT-4의 응답:

"신경학적 검사(neurological exam)와 함께,
필요시 MRI나 CT 스캔을 추천합니다.
또한 안과 검진도 고려하세요."

문제점:

1. **과도한 추천**: 모든 가능성을 다루려고 하여 불필요한 검사까지 권장 (비용 증가)
2. **진료과 혼동**: 신경과, 안과, 이비인후과 중 어디가 1차 진료과인지 불명확

[4] Accelerating Machine Learning Inference with Probabilistic Predicates 총정리

이 논문은 "비구조화 데이터에 대한 ML 추론 쿼리에서 비싼 UDF(User-Defined Function)를 어떻게 최적화할 것인가"라는 문제를 다룬다. Exqutor와는 다른 각도에서 ML+DB 최적화를 접근하지만, 두 논문 모두 **"ML/벡터 연산을 관계형 쿼리 옵티마이저에 어떻게 통합할 것인가"**라는 큰 질문을 공유한다.

핵심 아이디어는 비싼 ML 모델(딥러닝 기반 객체 탐지, 얼굴 인식 등) 이전에 **경량 이진 분류기를 "조기 필터"로 삽입**하는 것이다. 이를 통해 불필요한 데이터를 미리 걸러내고 비싼 ML 추론의 횟수를 크게 줄인다. 여러 필터를 계단식으로 연결(cascade)하면 각 단계에서 점진적으로 데이터가 정제되어 전체 비용을 극적으로 감소시킨다.

Exqutor가 벡터 검색의 **카디널리티**를 정확히 추정하여 옵티마이저를 돕는 것과 유사하게, 이 논문은 ML 추론의 **비용과 선택도**를 옵티마이저에 통합한다. 두 논문의 공통 메시지는 **"ML 연산의 특성을 옵티마이저에 정확히 반영하면 극적인 성능 향상이 가능하다"**는 것이다.

본 논문과의 관계; 비싼 연산(벡터 검색 vs. ML 추론)의 특성을 옵티마이저에 통합하는 서로 다른 접근법을 보여주며, 두 방식의 결합 가능성을 시사한다.

비디오 분석, 이미지 검색 같은 ML 추론 쿼리에서 핵심 병목은 **비싼 UDF(User-Defined Function)**이다.

```
SELECT * FROM video_frames
WHERE object_detector(frame, 'red SUV') = TRUE -- 매우 비쌌!
AND timestamp BETWEEN '2024-01-01' AND '2024-01-31';
```

여기서 `object_detector()` 는 딥러닝 모델로:

- ResNet 또는 YOLO 같은 대규모 신경망
- 한 프레임당 수십~수백 ms가 소요됨 (GPU 사용 기준)
- 100만 프레임 비디오라면; $100만 \times 100ms = 100,000초 = \text{약 } 28시간$ 소요

그런데 실제로 "빨간 SUV"가 나오는 프레임은 전체의 **0.1%도 안 될 수 있다** (예; 차가 10,000개 장면 중 10장에만 나타남).

[5] Analytical Engines with Context-Rich Processing: Towards Efficient Next-Generation Analytics 총정리

이 논문은 [6]번 논문(Context-Enhanced Relational Operators with Vector Embeddings)의 **전신이 되는 비전 논문**으로, 임베딩을 관계형 대수에 통합하는 아이디어의 출발점이다. 비전 논문(Vision Paper)이란 구체적인 실험보다는 **새로운 연구 방향을 제시하고 커뮤니티의 논의를 유도**하는 논문으로, [6]의 기술적 기여를 이해하기 위한 철학적·개념적 배경이 된다.

핵심 메시지는 현대 데이터의 **80~90%가 비구조화 데이터**인데, 관계형 DBMS는 여전히 구조화 테이블에 최적화되어 있다는 것이다. 따라서 **임베딩 연산자(E-Operator)**를 관계형 대수의 공식 연산자로 정의하면, 기존 최적화 규칙(선택 푸시다운, 조인 재정렬 등)을 임베딩 연산에도 자동으로 적용할 수 있다. 이것이 바로 VAQ(Vector-Augmented Analytical Query) 최적화의 이론적 토대가 된다.

본 논문과의 관계; 비전 논문으로서 구체적인 실험은 없지만, "임베딩을 관계형 대수에 통합하면 어떤 이득을 얻을 수 있을까"라는 근본적 질문에 대한 답을 제시하며, Exqutor가 이를 실행하는 데 필요한 정확한 카디널리티 추정의 필요성을 이론적으로 정당화한다.

과거(2010년대 초):

데이터 구성:

- 구조화: 80% (숫자, 날짜, 범주형)
- 비구조화: 20% (이미지, 텍스트)

도구 분포:

- RDBMS 투자: 80% (SQL, 트랜잭션, ACID)
- 비구조화 처리: 20% (특수 시스템)

현대(2020년대):

- 구조화: 10~20% (메타데이터만 해당)
- 비구조화: 80~90% (텍스트, 이미지, 오디오, 비디오)

도구 분포:

[6] Context-Enhanced Relational Operators with Vector Embeddings

이 논문은 벡터 임베딩을 관계형 데이터베이스 연산에 공식적으로 통합하는 이론적 기초를 제공한다. 핵심 기여는 **임베딩 연산자(E-Operator)**와 **컨텍스트 강화 조인(E-Join)**을 관계형 대수의 정식 연산자로 정의하고, 이를 기반으로 한 최적화 기법들을 제시하는 것이다.

논문의 중심 질문은 "구조화 데이터(숫자, 날짜)와 비구조화 데이터(텍스트, 이미지)를 같은 쿼리 안에서 의미 기반으로 분석하려면 어떻게 해야 하는가?"이다. 기존 SQL은 정확한 일치(exact match)나 범위 비교에만 적합했지만, 임베딩 연산자를 통합하면 "의미적으로 유사한" 데이터를 찾을 수 있게 된다.

핵심 기술 기여:

1. **프리페치 최적화:** 각 튜플의 임베딩을 한 번만 계산하여, 모델 호출 비용을 $O(|R| \times |S|)$ 에서 $O(|R| + |S|)$ 로 감소
2. **텐서 조인:** Nested Loop Join을 행렬 곱셈으로 변환하여 약 100배의 속도 향상 달성
3. **인덱스 vs. 스캔 분석:** 선택도에 따라 벡터 인덱스 사용의 효율성이 달라진다는 발견

본 논문과의 관계: Exqutor는 이 논문의 이론적 기초 위에서, "그렇다면 실행 계획을 세울 때 벡터 연산의 카디널리티를 어떻게 정확히 추정할 것인가?"라는 실용적 문제를 해결한다. 특히 인덱스 조인 vs. 스캔 조인의 선택이 선택도에 따라 달라진다는 이 논문의 발견은, Exqutor에서 정확한 카디널리티 추정이 왜 중요한지를 직접 입증한다.

현대 데이터 분석에서는 **구조화 데이터**(숫자, 날짜 등)와 **컨텍스트 풍부한 데이터**(텍스트, 이미지, 오디오 등)를 함께 분석해야 한다. 기존 관계형 연산자(SELECT, JOIN 등)는 정확한 일치(exact match)나 범위 비교에만 적합하고, "의미적으로 유사한" 데이터를 찾는 데는 부적합하다.

예를 들어, 두 테이블에서 "비슷한 상품 설명"을 기준으로 조인하고 싶다면:

```
-- 이런 쿼리는 기존 SQL로 표현 불가
SELECT * FROM table_a JOIN table_b
ON semantic_similarity(a.description, b.description) > 0.8
```

현재는 개발자가:

[7] SingleStore-V: An Integrated Vector Database System in SingleStore

이 논문의 핵심 기술 기여는 **Top() 물리적 연산자**로, ORDER BY + LIMIT 패턴을 인식하여 테이블 스캔 단계에서 직접 top-k 를 찾는 방식이다. 또한 **세그먼트 단위 인덱스 구조**를 통해 분산 환경에서의 확장성을 확보하고, **플러그형 벡터 인덱스**로 Faiss, hnswlib 등 다양한 인덱스를 동적으로 선택할 수 있도록 설계했다.

핵심 기술:

1. **Top() 연산자:** ORDER BY + LIMIT의 효율적 처리
2. **세그먼트 기반 인덱스:** 불변 세그먼트마다 독립적 인덱스, 병렬 검색
3. **플러그형 인덱스:** Faiss, HNSW 등 교체 가능한 인덱스 설계

본 논문과의 관계: SingleStore-V의 최적화는 주로 **단일 테이블 내 top-k 검색을 효율화**하는 데 집중한다. 반면 Exqutor는 **여러 테이블에 걸친 조인 순서, 조인 방식, 전체 실행 계획 최적화**에 집중한다. Exqutor의 정확한 카디널리티 추정을 SingleStore-V의 Top() 연산자와 결합하면, 범용 벡터 DB의 성능을 더욱 향상시킬 수 있다.

- **마스터-리프(Master-Leaf) 분산 구조:** 마스터 노드가 조정, 여러 리프 노드에서 데이터 저장 및 처리
- **행 저장소(Rowstore) + 열 저장소(Columnstore) 이중 저장:** 트랜잭션은 행 저장소에, 분석은 열 저장소에서 처리
- **불변 세그먼트(Immutable Segment):** 행 저장소의 데이터가 주기적으로 세그먼트화되어 열 저장소로 이동
- **샤딩(Sharding):** 데이터를 해시 키 기준으로 여러 리프 노드에 분산

SingleStore-V는 이 SingleStore에 벡터 검색 기능을 통합한 것으로, **범용 벡터 데이터베이스**의 대표 사례이다. Milvus 같은 전문 벡터 DB와 달리, 기존 SQL 쿼리와 벡터 검색을 하나의 SQL 문에서 처리할 수 있다.

SingleStore-V의 가장 중요한 기술적 기여는 **Top() 물리적 연산자**이다. 이것이 어떻게 문제를 해결하는지 이해하기 위해, 먼저 기존의 비효율적인 방식을 살펴보자.

기존 방식에서 top-k 벡터 검색을 SQL로 표현하면:

```
SELECT product_id, name, distance
```

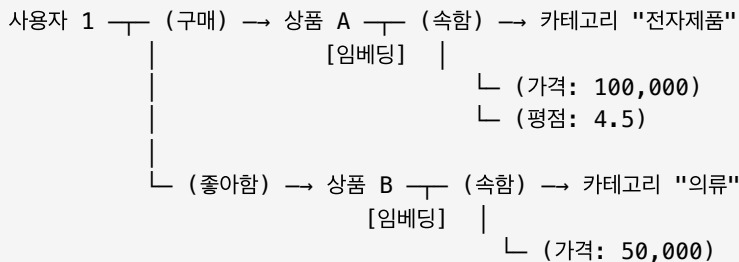
[8] High-Throughput Vector Similarity Search in Knowledge Graphs

지식 그래프는 현대 AI 응용의 핵심 인프라이다:

구조:

- **노드(Entities):** 상품, 사용자, 카테고리 등. 각 노드는 텍스트/이미지 임베딩을 가짐
- **엣지(Relations):** 노드 간 관계. 구조적 속성을 표현 (예: "상품 A는 카테고리 B에 속함")
- **속성(Attributes):** 가격, 평점, 설명 등의 메타데이터

예: 전자상거래 지식 그래프



[9] AnDB: Breaking Boundaries with an AI-Native Database for Universal Semantic Analysis

AnDB는 **AI 네이티브 데이터베이스**의 최신 동향을 보여주는 시스템으로, SQL 자체를 시맨틱 토큰으로 확장하여 비구조화 데이터에 대한 **선언적 시맨틱 분석**을 가능하게 한다.

핵심 아이디어는 전통적인 "SQL로 어떻게 벡터 검색을 표현할 것인가?"라는 문제에서 벗어나, "SQL 자체를 의미 기반 연산에 맞게 재설계하자"는 것이다. SQL에 3가지 시맨틱 토큰(PROMPT, SEM_MATCH, SEM_GROUP)을 추가하여, LLM 호출, 시맨틱 유사도 매칭, 시맨틱 그룹핑을 자연스럽게 표현할 수 있다.

핵심 기술:

1. **PROMPT 토큰**: SQL 내에서 LLM을 직접 호출
2. **SEM_MATCH 토큰**: 선언적 시맨틱 유사도 매칭
3. **SEM_GROUP 토큰**: 시맨틱 유사성 기반 그룹핑
4. **비용-정확도 옵티마이저**: LLM 호출 비용과 추론 정확도를 균형 맞춰 실행 계획 선택

본 논문과의 관계: Exqutor가 "기존 SQL에 벡터 검색을 추가"하는 보수적 접근이라면, AnDB는 "SQL 자체를 AI 연산에 맞게 재설계"하는 급진적 접근이다. AnDB 같은 시스템이 발전할수록, SEM_MATCH와 SEM_GROUP의 카디널리티를 정확히 추정해야 하므로, Exqutor 스타일의 카디널리티 추정이 **더욱 필요**해질 것이다.

현대 데이터는 **구조화 데이터(Structured)**와 **비구조화 데이터(Unstructured)**의 혼합이다:

구조화 데이터 (90년대 중심):

- 숫자: 매출, 수량, 나이
- 날짜: 주문 날짜, 생일
- 범주: 카테고리, 상태
- SQL로 정확히 분석 가능

비구조화 데이터 (현재의 90%):

[10] VBASE: Unifying Online Vector Similarity Search and Relational Queries via Relaxed Monotonicity

VBASE는 Exqutor가 직접 통합한 3개 시스템 중 하나이며, Exqutor 논문의 실험에서 **가장 극적인 성능 향상(최대 10,000 배)**을 보인 시스템이다. 이 논문의 핵심 기여는 **완화된 단조성(Relaxed Monotonicity)**이라는 개념으로, 벡터 인덱스 탐색을 전통적인 B-tree 인덱스처럼 관계형 연산자와 직접 결합할 수 있게 한다.

핵심 통찰:

- 전통적 DB 인덱스(B-tree)는 "다음 노드가 반드시 더 나쁠 것"이라는 **엄격한 단조성**을 만족
- 벡터 인덱스(HNSW)는 엄격한 단조성은 없지만, "충분히 탐색하면 더 좋은 결과가 나올 확률이 매우 낮아진다"는 **완화된 단조성**을 만족
- 이를 이용하면 TopK 고정값 없이도 동적으로 탐색을 종료할 수 있고, 벡터+관계형 쿼리를 Volcano 모델에 통합 가능

핵심 기술:

1. **완화된 단조성 검사**: 통계적 성질을 기반으로 탐색 종료 판정
2. **Volcano 모델 통합**: 벡터 인덱스를 표준 관계형 DB 실행 엔진에 직접 삽입
3. **필터링된 유사도 검색**: 벡터 조건 + 관계형 필터를 한 번에 처리

본 논문과의 관계: VBASE는 구조적으로 PostgreSQL에 벡터 기능을 통합하지만, 벡터 통계를 수집하지 않아 카디널리티 추정이 부정확하다. Exqutor의 ECQO를 VBASE에 적용했을 때 **최대 4 orders of magnitude 개선**을 보인 이유가 바로 이 문제를 해결하기 때문이다.

단조성이란? 데이터 구조를 탐색할 때, "다음에 방문할 데이터가 현재보다 '더 나쁜' 결과를 줄 것이라면, 탐색을 멈춰도 된다"는 성질이다.

전통적인 관계형 DB의 B-tree 인덱스는 이 단조성을 **완벽히 만족한다**:

예: 가격이 < 1000인 상품 찾기
B-tree 인덱스 (가격순 정렬):

[11] Milvus: A Purpose-Built Vector Data Management System

- 1. 사용하기 쉬운 인터페이스:** 다양한 응용 분야의 개발자가 쉽게 벡터 검색을 활용할 수 있도록 파이썬, Java, Go 등 여러 언어의 SDK를 제공한다. 개발자는 복잡한 인덱스 구조를 이해할 필요 없이, 간단한 API(search, insert, delete)만으로 벡터 검색 시스템을 구축할 수 있다.
 - 2. 이기종 컴퓨팅 최적화:** CPU와 GPU를 효율적으로 활용하여 성능을 극대화한다. 벡터 검색은 높은 병렬성을 가지므로, GPU의 수천 개 코어를 활용하면 성능을 10배 이상 향상시킬 수 있다. Milvus는 이러한 GPU 가속을 자동으로 처리하므로, 개발자는 GPU 프로그래밍 없이도 GPU의 이점을 누릴 수 있다.
 - 3. 단순 유사도 검색을 넘어선 고급 쿼리:** 속성 필터링(메타데이터 조건), 다중 벡터 검색(여러 벡터 필드 동시 검색), 범위 검색(거리 임계값) 등 실무에서 필요한 고급 기능을 지원한다.
 - 4. 동적 데이터 관리:** 실시간으로 데이터가 삽입·수정·삭제되면서 동시에 검색 요청을 처리해야 하는 상황을 안정적으로 지원한다. 인덱스를 재구축하지 않으면서도 새로운 데이터를 즉시 검색 가능하게 만드는 것이 핵심이다.
 - 5. 확장성과 가용성:** 분산 환경에서 데이터와 쿼리 처리를 여러 노드에 분산시켜, 수십억 건의 벡터를 저장·검색할 수 있어야 한다. 동시에 노드 장애 시에도 서비스가 중단되지 않도록 복제(replication)를 지원해야 한다.
- Milvus는 **클라우드 네이티브 아키텍처**를 채택하여 다음 특징을 갖는다.

저장과 연산의 분리:

- 6. 저장소:** Amazon S3, Google Cloud Storage 같은 객체 스토리지를 사용하여 고가용성과 확장성을 확보한다. 데이터는 스토리지에 영구적으로 저장되며, 여러 노드에서 동시에 접근할 수 있다.
- 7. 연산:** 상태 없는(stateless) 컴포넌트들이 탄력적으로 확장·축소된다. 쿼리 처리 노드는 필요에 따라 동적으로 추가하거나 제거할 수 있으므로, 트래픽 변동에 자동으로 대응할 수 있다.

세그먼트 기반 데이터 관리:

- 8. 데이터는 컬렉션(Collection) 단위로 관리되며,** 이는 관계형 DB의 테이블에 해당한다.
- 9. 컬렉션은 여러 세그먼트(Segment)로 나뉘며,** 이는 물리적 저장 단위이다. 각 세그먼트는 불변(immutable)이므로, 한번 생성되면 수정되지 않는다.
- 10. 세그먼트 단위로 독립적인 인덱스를 구축하고 검색을 수행한다.** 새 데이터가 들어오면 새 세그먼트가 생성되고, 백그라운드에서 정기적으로 여러 세그먼트를 병합한다.

조절 가능한 일관성(Tunable Consistency):

- 11. 로그 백본(log backbone):** 모든 쓰기 작업을 순서대로 로그에 기록하여, 분산 시스템에서 데이터 일관성을 관리한다.
- 12. 사용자는 일관성-성능 트레이드오프를 제어할 수 있다:**

[12] Qdrant: High-Performance Vector Search at Scale

Qdrant의 가장 핵심적인 기술적 기여는 벡터 유사도 검색(HNSW)과 메타데이터 필터링을 **동시에** 수행하는 구조이다.


```

client.search(
    collection_name="products",
    query_vector=[0.1, 0.2, ...],
    query_filter=Filter(
        must=[FieldCondition(key="price", range=Range(lt=1000))]
    ),
    limit=10
)

```

필터 선택도에 따른 자동 전략 전환:

Qdrant의 핵심 통찰은 필터의 선택도(어느 정도 비율의 데이터가 필터를 통과하는가)에 따라 최적의 검색 전략이 달라진다는 것이다.

1. **선택도 높음** (결과 많음, 예: 80%의 데이터가 조건을 만족):
 - HNSW 그래프를 탐색하면서 조건을 불만족하는 노드를 건너뛴
 - 거의 모든 데이터가 조건을 만족하므로, 필터를 먼저 적용하는 것이 비효율적

[13] pgvector: Open-Source Vector Similarity Search for PostgreSQL

왜 이것이 문제인가?

PostgreSQL의 옵티마이저는 각 조건의 선택도(결과의 몇 %가 그 조건을 통과하는가)를 기반으로 **조인 순서, 조인 방식, 인덱스 사용 여부**를 결정한다.

예시 쿼리:

```

SELECT * FROM items
WHERE (embedding <=> query_vector) < 0.1  -- 벡터 조건
    AND category = 'electronics'          -- 스칼라 조건
ORDER BY price;

```

시나리오 1: 벡터 조건의 실제 선택도 = 0.1% (1000개 중 1개만 유사)

- 옵티마이저의 추정: 33.3%
- 실제 결과: 0.1%

이 경우 옵티마이저는 **HNSW 인덱스를 사용하지 않고 Sequential Scan**을 선택할 수 있다 (예상 결과가 많으니 풀 스캔이 빠르다고 판단). 하지만 실제로는 인덱스를 사용하는 것이 훨씬 빠르다.

시나리오 2: 벡터 조건의 실제 선택도 = 90% (거의 모든 데이터가 유사)

- 옵티마이저의 추정: 33.3%
- 실제 결과: 90%

[14] ClickHouse: Lightning Fast Analytics for Everyone

ClickHouse도 범용 DB에 벡터 검색을 추가한 경우로, 카디널리티 추정 문제를 안고 있을 것으로 예상된다. Exqutor의 ECQO 개념을 ClickHouse에 적용하면 유사한 성능 향상을 기대할 수 있다.

기존 OLAP 엔진의 딜레마:

대부분의 데이터베이스는 다음 두 요구사항 중 하나에 특화된다:

1. OLTP (Online Transactional Processing):

- 소량의 데이터를 빠르게 읽고 쓰기 (예: 주문 삽입, 사용자 조회)
- 트랜잭션 안전성, 동시성 제어 중심
- 예: MySQL, PostgreSQL

2. OLAP (Online Analytical Processing):

- 대량의 데이터를 복잡하게 분석 (예: 월간 판매액 집계, 고객 세분화)
- 쿼리 속도, I/O 효율성 중심
- 예: 기존 Redshift, BigQuery

ClickHouse는 "두 마리 토끼를 모두 잡겠다"는 야망을 가지고 설계되었다:

ClickHouse의 목표:

- **고속 적재**: 초당 수백만 건의 데이터를 스트리밍으로 받아서 적재
- **저지연 분석**: 분석 쿼리를 초 단위(또는 밀리초 단위)로 실행

[15] Blended RAG: Improving RAG Accuracy with Semantic Search and Hybrid Query-Based Retrievers

RAG(Retrieval-Augmented Generation)란?

RAG는 다음 절차로 동작한다:

사용자 질문 → 검색 (관련 문서 찾기) → LLM 입력 (검색 결과 + 질문) → 생성된 답변

검색 단계가 LLM 응답 품질의 상한선(ceiling)을 결정한다:

- 검색 결과가 정확하면 → LLM이 좋은 답변 생성 가능
- 검색 결과가 부정확하거나 불안정하면 → LLM이 아무리 좋아도 제한된 답변만 가능

단일 검색 전략의 한계:

기존 RAG 시스템은 한 가지 검색 방식만 사용했다:

1. 키워드 검색(BM25)만 사용:

- 장점: 정확한 단어 매칭에 강함 (예: "Apple Inc." 검색 시 "Apple" 회사만 찾을 수 있음)
- 단점: 의미적 유사성을 못 잡음
- 질문: "자동차 사고"
- 찾지 못하는 문서: "교통 충돌", "차량 사건" (의미가 같지만 단어가 다름)

2. 밀집 벡터 검색(KNN)만 사용:

[16] Scalable Similarity Search for Big Data

핵심 기여:

1. **메트릭 공간의 기하학적 성질**: 고차원 공간에서의 거리 분포 특성 분석
2. **M-tree 인덱스 구조**: 메트릭 공간의 분할 기반 동적 인덱스
3. **고차원의 저주와 카디널리티**: 거리 집중(concentration) 현상의 수학적 분석
4. **분산 환경으로의 확장**: 빅데이터 환경에서의 유사도 검색

이 논문은 Exqutor가 통계 기반이 아닌 실제 인덱스 탐색 기반의 카디널리티 추정을 선택한 이유의 이론적 근거를 제공한다.

세 가지 핵심 도전 과제:

1. 확장성(Scalability):

- 데이터가 수십억 건에서 수조 건으로 증가해도 응답 시간이 허용 범위 내여야 한다.
- 기존 풀 스캔(brute-force) 방식은 $O(n)$ 복잡도이므로, 데이터 크기 n 이 1000배 증가하면 응답 시간도 1000배 증가한다. 이는 참을 수 없다.
- 인덱스를 통해 $O(\log n)$ 또는 $O(\log^k n)$ 수준으로 개선해야 한다.

2. 프라이버시 보존(Privacy Preservation):

- 클라우드 아웃소싱 환경에서 원본 데이터는 공개하지 않으면서 유사도 검색을 수행해야 한다.
- 암호화된 데이터에 대한 검색(searchable encryption) 기술이 필요하다.

3. 멀티모달 통합(Multimodal Integration):

[17] Spider 2.0: Evaluating Language Models on Real-World Enterprise Text-to-SQL Workflows

Spider 1.0의 문제점:

기존 Spider 1.0 벤치마크는 학술 환경의 편의성을 위해 다음과 같이 단순화되어 있었다:

| 특성 | Spider 1.0 | 현실 기업 데이터베이스 |

|-----|-----|-----|

| 테이블 수 | 평균 5~10개 | 수백~수천 개 (대기업은 1만+ 테이블) |

| 컬럼 수 | 평균 ~20개 | 1,000개 이상 (한 테이블만 500개 컬럼) |

| SQL 방언 | SQLite만 | BigQuery, Snowflake, PostgreSQL, T-SQL, Oracle 등 다양 |

| 쿼리 복잡도 | 단일, 간단한 쿼리 | 100줄 이상의 멀티 쿼리 워크플로우 |

| 전처리/후처리 | 불필요 | CRITICAL: CLI 호출, 파일 변환, API 통합 |

| 도메인 지식 | 기본 SQL | 도메인별 비즈니스 로직 필수 |

실제 예시 쿼리:

```
-- 실제 기업 쿼리 (Spider 2.0)
WITH sales_data AS (
  SELECT
    order_id,
```

[18] An Efficient and Robust Framework for Approximate Nearest Neighbor Search with Attribute Constraint

이 논문은 벡터 검색과 속성 필터를 동시에 처리하는 효율적인 하이브리드 쿼리 프레임워크 NHQ(Native Hybrid Query)를 제안한다. 기존의 Pre-filtering (필터 먼저 적용) 또는 Post-filtering (검색 먼저 수행) 접근법은 속성 선택도가 낮거나 높을 때 극단적인 성능 저하를 보이는 반면, NHQ는 Navigable Proximity Graph(NPG) 구조를 통해 벡터 근접성과 속성 제약을 동시에 활용함으로써 최대 315배의 성능 향상을 달성한다. 특히 속성 분포의 균일성 여부에 따라 두 가지 변형(NPG-C와 NPG-A)을 제공하여, 다양한 실세계 데이터셋에서 재현율 90% 이상을 유지하면서 높은 효율성을 보장한다. NeurIPS 2023의 발표로 학술적 공신력이 높으며, 필터링된 벡터 검색 분야의 최신 기술을 대표한다.

현대의 데이터 검색 애플리케이션은 단순한 벡터 유사도 검색만으로는 부족하다. 대부분의 실세계 시나리오에서는 벡터 근접성(vector proximity)과 속성 제약(attribute constraints)을 함께 처리해야 한다. 예를 들어, 이미지 검색 시스템에서는 "이 이미지와 비슷한 이미지를 찾되, 특정 카메라 모델로 촬영된 것만 원한다"는 요구가 흔하다. 문서 검색에서는 "이 텍스트와 의미가 비슷한 문서를 찾되, 특정 시간 범위와 카테고리만 제한하고 싶다"는 다중 필터가 일반적이다. 이런 하이브리드 쿼리(HQ) 환경에서 기존의 두 가지 접근법은 근본적인 한계를 보인다:

1. 속성 조건을 먼저 적용: WHERE category = 'electronics'
2. 필터링 후 남은 데이터에 대해 ANN 검색 수행

장점:

- 검색 범위가 작아지므로, 작은 필터링 결과 집합에 대해서는 빠르다.

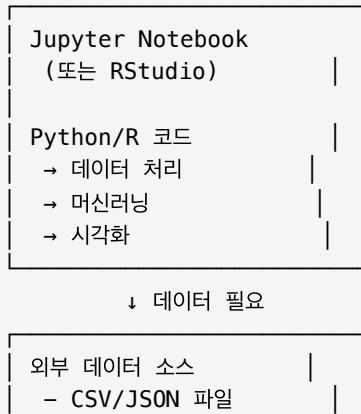
치명적 문제점:

- 속성 조건의 선택도가 낮으면 (예: 1% 미만의 데이터만 통과), 필터링 후 남은 데이터 양이 극소이다.
- 이 극소 데이터셋에 대해서는 벡터 인덱스(예: HNSW)가 효과적으로 작동하지 않는다. 인덱스는 일정 규모 이상의 데이터를 가정하고 설계되었기 때문이다.
- 결과적으로 순차 탐색(linear scan)으로 되돌아가야 하며, 이는 대규모 원본 데이터의 작은 부분만 건너뛰는 것이므로 오버헤드가 크다.
- 또한 별도의 부분 인덱스(partial index)를 구축해야 하므로, 저장 공간과 인덱스 유지 비용이 증가한다.

[19] DuckDB: An Embeddable Analytical Database

현대의 데이터 분석가와 과학자들(주로 파이썬/R 사용자)은 다음과 같은 환경에서 작업한다:

분석 환경:



[20] Are There Fundamental Limitations in Supporting Vector Data Management in Relational Databases? A Case Study of PostgreSQL

역할군: (E) 문제 분석

2. No, 단순 구현 문제다

- 현재 구현이 미흡할 뿐, 개선으로 충분하다
- 벡터 지원은 PostgreSQL에 완벽히 통합될 수 있다

3. Yes and No, 둘 다다

- 일부는 근본적 제약(버퍼 풀, MVCC)
- 일부는 개선 가능(통계, 인덱스 최적화)

이 논문의 대답: 3번. "일부는 근본, 일부는 구현"

문제점: 래치(Latch) 경합

HNSW 탐색의 메모리 접근 패턴:

- 무작위 접근: 순차적이지 않음
- 페이지 접근 빈도: 매우 높음 (초당 수천~수만 번)

PostgreSQL의 버퍼 풀 동시성 제어:

- 각 페이지마다 래치 존재
- 페이지 읽기 전: 래치 획득 (spin lock)

[21] DNA Sequence Classification with Milvus

- 결과 해석 가능 (k-mer 기여도 확인)

1. 쿼리 DNA 서열 입력 |
 예: 새로운 박테리아 서열 |

↓

2. 벡터 변환 |
 - k-mer 추출 |
 - DNABERT 임베딩 |
 - 결과: 1536D 벡터 |

↓

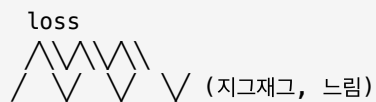
[22] On the Importance of Initialization and Momentum in Deep Learning

신경망을 학습할 때 **확률적 경사 하강법(SGD)**이 느리게 수렴하는 현상이 있다:

이상적 수렴:



실제 SGD 수렴:



원인: 손실 함수의 지형이 방향마다 다르다

[23] New TPC Benchmarks for Decision Support and Web Commerce

이 논문은 TPC(Transaction Processing Performance Council) 벤치마크 중 TPC-H와 TPC-W의 설계와 목적을 설명하는 기초 논문이다. 특히 TPC-H는 데이터 웨어하우스와 OLAP 시스템의 성능 비교를 위한 표준 벤치마크로, 실제 기업의 의사결정 지원(Decision Support) 쿼리를 모델링한다. Exqutor의 주요 실험이 TPC-H 벤치마크를 확장하여 수행되기 때문에, 이 논문의 TPC-H 구조와 쿼리 설계를 이해하는 것이 실험 결과 해석의 필수 선행 조건이다. 벤치마크 설계의 정당성과 데이터베이스 성능 평가의 원칙을 이해하는 데 중요하다.

문제: 데이터베이스 벤더들이 마음대로 성능 주장

예시 (1990년대):

Oracle: "우리 DB가 가장 빠르다"

SQL Server: "우리가 더 빠르다"

PostgreSQL: "우리가 최고다"

증거?

각자 자신에게 유리한 테스트만 수행

다른 DB와 비교하지 않음

테스트 조건을 공개하지 않음

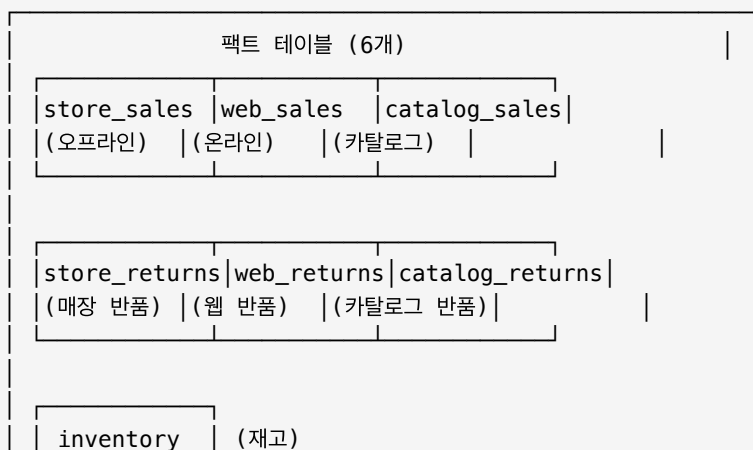
결과:

구매자는 어떤 DB가 실제로 좋은지 알 수 없음

성능 비교가 "입씨름" 수준

[24] The Making of TPC-DS

핵심: 스타/스노우플레이크 스키마



[25] Vector database management techniques and systems 총정리

벡터 데이터베이스는 단순한 숫자 배열 저장소가 아닌 복잡한 관리 시스템이다. 전통적 관계형 데이터베이스와 달리, 벡터 DB는 다음의 고유한 문제점들을 직면한다:

- **고차원성 저주 (Curse of Dimensionality)**: 수천 또는 수만 차원의 벡터 데이터에서 정확한 최근접 이웃 검색은 계산 비용이 폭발적으로 증가
- **정확성 vs 속도 트레이드오프**: 완전히 정확한 검색은 실시간 성능을 해치고, 근사 알고리즘은 결과의 정확성을 감소
- **이질성 데이터 관리**: 벡터와 메타데이터의 결합 관리, 하이브리드 쿼리 처리
- **동적 데이터 변화**: 삽입, 삭제, 업데이트 작업 중 인덱스 유지보수 오버헤드
- **메모리 효율성**: 고차원 벡터의 메모리 소비가 대규모 데이터셋에서 심각

2. **트리 기반 인덱싱**: KD-트리, Ball 트리 등 계층적 공간 분할 구조

3. **그래프 기반 인덱싱**: HNSW(Hierarchical Navigable Small World) 같은 근치이웃 그래프 구조

4. **양자화 기법 (Quantization)**: 벡터를 저정밀 표현으로 압축하여 메모리와 속도 향상

고차원 공간에서 정확한 최근접 이웃 검색의 NP-hard 특성을 완화하기 위한 근사 알고리즘:

- **Hierarchical Clustering**: 데이터를 계층적으로 클러스터링하여 검색 공간 축소
- **Random Projection**: 고차원 데이터를 저차원으로 사영
- **Product Quantization**: 벡터를 부분 벡터로 분해하여 양자화
- **Graph Traversal**: 사전 구성된 그래프를 통한 탐욕적 이웃 탐색

2. **인덱싱 계층**: 다양한 인덱스 구조의 생성, 유지, 쿼리 처리

3. **쿼리 처리 엔진**: SQL 파싱, 최적화, 실행 계획 수립

[26] Pinecone: The vector database to build knowledgeable AI 총정리

벡터 데이터베이스의 수요 증가에 따른 여러 실무적 문제들:

- **운영 복잡성**: 오픈소스 벡터 DB를 직접 운영할 경우 배포, 확장, 백업, 모니터링의 복잡성
- **확장성 문제**: 소규모에서는 간단하지만, 대규모 벡터 데이터셋에서의 성능 유지의 어려움
- **일관성 보장**: 분산 환경에서 데이터 복제 및 일관성 유지
- **보안 및 접근 제어**: 엔터프라이즈 환경에서의 인증, 권한 관리, 데이터 암호화
- **메타데이터 통합**: 벡터와 함께 저장된 메타데이터의 효율적 필터링 및 검색
- **높은 초기 비용**: 전문 엔지니어링 팀 구성, 하드웨어 투자

클라이언트 애플리케이션

↓

Pinecone API (REST/gRPC)

↓

Query Router (요청 분배)

↓

인덱싱 엔진 (복수 Pod)

↓

[27] Manu: A cloud native vector database management system 총정리

클라우드 환경에서의 벡터 데이터베이스 운영이 마주하는 특유한 도전 과제들:

- **리소스 변동성**: 클라우드에서 계산 리소스가 동적으로 증감하는 환경에서 데이터 일관성과 가용성 유지
- **네트워크 지연**: 마이크로서비스 아키텍처에서의 서비스 간 통신 오버헤드
- **확장성 제약**: 전통적 중앙집중식 아키텍처의 성능 병목
- **상태 관리의 복잡성**: 분산된 노드들 간의 벡터 데이터와 인덱스 상태 동기화
- **자동 장애 복구**: 클라우드 환경에서의 예측 불가능한 장애 대응
- **효율적 리소스 활용**: 비용 절감과 성능 사이의 균형

1. **메타데이터 Etcid 병목**: 모든 메타데이터 변경이 Etcid를 거쳐야 할 때, Etcid의 처리 능력 한계는? 분산 환경에서 메타데이터 캐싱의 일관성 보장 수준은?
2. **세그먼트 병합 오버헤드**: 대규모 데이터셋에서 주기적인 세그먼트 병합이 쿼리 성능에 미치는 영향을 측정하고 최소화할 수 있는가?
3. **캐시 무효화 전략**: 분산 검색 노드들 간의 캐시 일관성을 어느 수준까지 자동으로 유지할 수 있는가? 수동 개입이 필요한 경우는?
4. **인덱스 선택 자동화**: 다양한 인덱싱 알고리즘(IVF, HNSW, DiskAnn) 중 최적을 선택하는 기준과 비용-성능 분석은?
5. **장애 복구 시간**: 검색 노드 장애 시 페일오버 시간과, 저장 노드 장애 시 데이터 복구 시간의 SLA는?

[28] Chroma: The open-source AI application database 총정리

기존 벡터 데이터베이스 솔루션의 한계와 Chroma의 해결 방안:

- **복잡성**: Milvus, Weaviate 등 기존 시스템의 설정과 배포가 복잡
- **Chroma 해결책**: 최소 구성으로 5분 내 시작 가능
- **진입 장벽**: 소규모 팀이나 초기 단계 스타트업의 높은 인프라 비용
- **Chroma 해결책**: 로컬 개발 모드 완전 무료, 메모리 기반 배포
- **프로토타이핑 속도**: 빠른 개념 검증의 필요성
- **Chroma 해결책**: 즉시 사용 가능한 Python API
- **확장성 필요**: 개발 중 쉽게 확장할 수 있는 아키텍처
- **Chroma 해결책**: 로컬 → 분산 모드로의 무경계 전환

1. **확장성 한계**: SQLite/DuckDB 기반의 메타데이터 저장소가 어느 정도의 벡터 개수와 쿼리 빈도까지 효율적으로 처리할 수 있는가?
2. **정확한 vs 근사 검색의 자동 선택**: 컬렉션 크기에 따라 자동으로 인덱싱 전략을 전환하는 기준은? 전환 과정의 오버헤드는?
3. **멀티 클라이언트 동시성**: 클라이언트-서버 모드에서 다양한 클라이언트의 동시 삽입/삭제/검색 시 일관성 보장 수준은?
4. **메모리 효율성**: 인메모리 모드에서 대규모 벡터 데이터셋(수억 개)을 저장할 때 메모리 압축 기법은?
5. **분산 모드 성능**: 분산 모드에서 네트워크 지연이 단일 노드 모드 대비 검색 성능에 얼마나 영향을 미치는가?
6. **임베딩 모델 버전 관리**: 임베딩 모델이 변경될 때 기존 벡터의 재생성과 재인덱싱 전략은?

[29] Survey of vector database management systems 총정리

벡터 데이터베이스 관리 시스템의 포괄적이고 최신의 서베이 논문이다. 이 논문은 벡터 DB의 탄생 배경, 발전 역사, 현재의 다양한 시스템들을 체계적으로 분류하고 분석한다. 벡터 DB의 핵심 기술 영역인 인덱싱, 근사 최근접 이웃 검색, 메타데이터 관리, 분산 처리, 그리고 최적화 기법들을 상세히 다룬다. VLDB 저널의 게재를 통해 데이터베이스 커뮤니티의 공식적 인정을 받은 이 서베이는, 벡터 DB 관련 최신 연구 동향을 정리하고, 해결되지 않은 문제들을 지적하며, 향후 연구 방향을 제시한다. 약 600개 이상의 참고문헌을 포함하여, 벡터 DB 분야의 가장 종합적이고 신뢰할 수 있는 기술 참고자료이다.

- 관계형 DB: 구조화된 데이터에 최적화, 벡터 데이터 처리 부실
- 문서 DB: 반구조화 데이터에 강하나 의미적 유사도 검색 미지원

AI 혁명의 영향:

- 딥러닝의 급속한 발전으로 특성 추출(feature extraction)이 표준화
- 대규모 텍스트 임베딩 모델의 등장 (Word2Vec, BERT, GPT 기반 임베딩)
- LLM 기반 애플리케이션의 외부 지식 베이스 필요성

벡터 DB의 필연성:

- 수백만 수억 차원의 벡터 데이터를 효율적으로 관리할 수 있는 전문 시스템의 부재
- 전통 DB에 벡터 지원 추가의 복잡성과 성능 한계
- HNSW (Hierarchical Navigable Small World): 다층 그래프 구조, 빠른 검색
- IVF (Inverted File Index): 클러스터 기반, 효율적 메모리 사용
- LSH (Locality Sensitive Hashing): 확률적 기법, 병렬 처리 용이

디스크 기반 인덱싱:

- DiskANN: 메모리 기반 그래프와 디스크 저장의 결합

[30] Pase: PostgreSQL ultra-high-dimensional approximate nearest neighbor search extension 총 정리

PostgreSQL과 같은 기존 RDBMS에 벡터 검색을 추가할 때의 도전 과제:

- **고차원 데이터의 성능 병목:** 수천 차원의 벡터에 대해 정확한 최근접 이웃 검색은 전체 테이블 스캔을 필요로 하며, 초고차원에서는 "차원의 저주" 현상으로 근사도 어려움
- **인덱싱 구조 부재:** B-tree, GiST 등 기존 인덱싱 기법은 고차원 공간에 부적합
- **저장소 확장:** 기존 RDBMS의 행(row) 기반 저장소에 대규모 벡터를 효율적으로 저장
- **쿼리 최적화:** 벡터 조건과 스칼라 조건을 포함한 복합 쿼리의 최적화
- **정확도-성능 트레이드오프:** 근사 알고리즘 사용 시 발생하는 정확도 손실 관리

이진 트리 구조로 조직화

특징:

- 트리 깊이 $O(\log n)$: 로그 시간에 쿼리 벡터가 속한 영역 결정
- 메모리 효율적: 각 노드는 분할 평면만 저장
- 적응형 분할: 데이터 분포에 따른 자동 조정

****후보 확대 (Expansion Strategy)**:**

쿼리 벡터의 영역부터 시작

↓

[31] DuckDB-VSS: Vector similarity search extension for DuckDB 총정리

기존 벡터 검색 솔루션의 한계:

- **분석과 벡터 검색의 분리**: 기존 벡터 DB는 순수 벡터 검색에만 최적화, 정형 데이터와의 복합 분석이 어려움
- **OLAP 환경의 필요성**: 분석가가 벡터와 정형 데이터를 함께 분석해야 하는 경우 증가
- **진입 장벽**: 별도의 벡터 DB 시스템 관리의 복잡성
- **비용**: 중규모 조직이 벡터 DB를 위해 추가 인프라 투자의 부담
- **컬럼 저장소의 미활용**: DuckDB의 효율적 컬럼 저장소를 벡터 데이터에도 활용하지 못함

1. **메모리 제약 극복**: 인메모리 시스템의 근본적 한계를 어떻게 극복할 것인가? 디스크 기반 벡터 저장에 성능에 미치는 영향은?
2. **HNSW 파라미터 자동화**: `m` 과 `ef` 같은 파라미터를 사용자가 수동으로 튜닝해야 하는데, 자동 최적화 방법은?
3. **분산 처리**: DuckDB의 장점을 유지하면서 분산 벡터 검색을 지원할 수 있는가? 단일 노드 제약을 어떻게 해결할 것인가?
4. **인덱스 유지보수**: 높은 빈도의 삽입 작업 중 HNSW 인덱스의 지속적 갱신이 성능에 미치는 영향은?
5. **다양한 벡터 차원 지원**: 모든 차원의 벡터에 대해 일정한 성능을 유지할 수 있는가? 극단적으로 높은 차원(100,000+)의 벡터는 지원 가능한가?
6. **비교 분석**: DuckDB-VSS와 Pinecone, Milvus 같은 전문 벡터 DB의 성능-비용 트레이드오프는?

[32] Vespa: AI + data, online at any scale 총정리

Vespa는 Yahoo에서 개발한 오픈소스 검색 및 AI 데이터 서빙 플랫폼이다. 단순한 벡터 DB를 넘어, 구조화된 데이터, 벡터, 그리고 복잡한 추론 로직을 모두 포괄하는 포괄적 시스템이다. Vespa의 핵심 특징은 실시간 데이터 업데이트와 높은 처리량의 개인화된 검색/추천을 동시에 제공한다는 점이다. 대규모 인터넷 서비스에서 사용되었던 검색 및 추천 엔진의 경험을 바탕으로, 프로덕션급 신뢰성과 확장성을 갖추고 있다. 텍스트 검색, 벡터 검색, 정형 데이터 필터링, 그리고 머신러닝 모델의 추론을 하나의 통일된 플랫폼에서 처리하므로, 현대적 AI 애플리케이션에 매우 적합하다.

기존 검색/추천 시스템의 한계:

- 오프라인-온라인 갭: 오프라인에서 학습한 모델을 온라인 시스템에 배포할 때의 복잡성과 지연
- 실시간 개인화의 어려움: 사용자의 최근 행동을 반영한 즉시 개인화 추천의 기술적 복잡성
- 다양한 데이터 타입 통합: 정형 데이터, 텍스트, 벡터, 그래프 등을 통일된 방식으로 처리
- 확장성과 지연시간: 초당 수백만 요청을 수백 밀리초 이내에 처리
- AI 모델 배포: 새로운 ML 모델의 빠른 반영과 버전 관리

Vespa의 해결책:

- 통일된 플랫폼으로 모든 데이터와 로직을 처리
- 실시간 데이터 피드 처리
- 쿼리 시 동적 추론 실행
- 자동 확장과 고가용성

클라이언트 애플리케이션



[33] Neo4j: Vector index and search 총정리

Neo4j는 그래프 데이터베이스로 잘 알려져 있으며, 최근 벡터 인덱싱과 검색 기능을 핵심 플랫폼에 통합했다. Neo4j의 벡터 검색은 단순한 임베딩 저장이 아닌, 그래프의 노드와 관계와 함께 벡터 데이터를 저장하고 검색하는 통합된 접근이다. 이를 통해 의미적 유사도(벡터 기반)와 구조적 유사도(그래프 관계 기반)를 함께 고려하는 강력한 검색이 가능해진다. 지식 그래프, 추천 시스템, 의미 검색 등 다양한 응용에서 그래프 구조와 의미적 정보를 동시에 활용할 수 있다.

그래프 데이터베이스 환경에서 벡터 검색의 필요성:

- **의미적 검색의 부재**: 기존 그래프 DB는 명시적 관계와 속성 기반 검색만 지원, 의미적 유사도 검색 불가
- **구조와 의미의 분리**: 관계 정보와 벡터 정보를 별도 시스템에서 관리해야 했던 문제
- **지식 그래프의 완성도**: 엔티티 간 벡터 기반 유사도를 고려한 검색 필요
- **추천 시스템의 통합**: 사용자-아이템 관계와 벡터 유사도를 함께 고려하는 하이브리드 추천
- **오버헤드**: 그래프와 벡터 DB를 동시에 운영하는 복잡성과 데이터 동기화

Neo4j의 해결책:

- 벡터를 노드의 속성으로 저장
- HNSW 기반 벡터 인덱싱
- 그래프 탐색과 벡터 검색의 통합 쿼리
- 관계 기반 검색에 벡터 필터 추가 가능
- `m`: HNSW 그래프 구조의 파라미터 (연결 수)
- `efConstruction`: 인덱스 생성 시 탐색 범위
- `ef`: 쿼리 시 탐색 범위

[34] Redis: for vector database 총정리

Redis는 초고속 인메모리 데이터 구조 저장소로서, RedisVectorDB 모듈과 RediSearch의 벡터 기능을 통해 벡터 검색 능력을 갖추고 있다. Redis의 핵심 강점은 극도로 빠른 응답 시간(서브 밀리초)과 높은 처리량(초당 수백만 작업)이다. 캐싱, 세션 저장소로 널리 사용되는 Redis의 신뢰성과 생태계를 활용하면서, 벡터 검색이 필요한 실시간 애플리케이션에 완벽하게 맞다. Redis의 벡터 검색 기능은 Pinecone이나 Milvus 같은 전문 벡터 DB에 비해 배포가 간단하고, 기존 Redis 사용자들에게는 추가 인프라 없이 벡터 검색을 추가할 수 있다.

기존 벡터 검색 솔루션의 한계:

- **초저지연 요구**: 실시간 추천, 자동완성 등에서 밀리초 단위 응답 필요, 기존 벡터 DB는 충분하지 않을 수 있음
- **인프라 복잡성**: 추가 벡터 DB 시스템 운영의 부담
- **기존 Redis 활용 미흡**: 많은 기업이 이미 Redis를 사용하고 있으나, 벡터 지원이 제한적
- **데이터 동기화**: 캐시와 벡터 DB를 별도로 운영할 때의 동기화 복잡성
- **높은 초기 비용**: 새로운 전문 시스템 도입의 학습곡선과 비용

Redis의 해결책:

- 기존 Redis 인프라 위에서 벡터 검색
- 극도로 빠른 응답
- 간단한 배포와 운영
- 캐싱과 벡터 검색의 통합
- **M**: HNSW 그래프의 구조 파라미터 (클수록 정확하나 메모리 사용 증가)
- **EF_CONSTRUCTION**: 인덱싱 시간의 정확도 (클수록 정확하나 느림)
- **EF_RUNTIME**: 쿼리 지연시간 vs 정확도 (클수록 정확하나 느림)

[35] K-nearest neighbor

논문에서는 KNN의 기본 원리, 거리 측정 방식, 최적 K값 선택 방법, 그리고 계산 효율성 개선 방안을 다룬다. 특히 유클리드 거리, 맨해튼 거리 등 다양한 거리 메트릭과 그들의 특성을 설명한다. 또한 KNN의 장점으로 구현의 단순성, 비모수 성질, 그리고 다양한 문제에 대한 적응성을 강조한다.

본 논문과의 관계: Exqutor 시스템은 벡터 기반의 유사도 검색을 위해 KNN 개념을 확장한다. 본 논문은 KNN의 기본적인 이론적 배경을 제공하며, Exqutor가 구현하는 필터링 기반 근접 이웃 검색의 근본이 되는 개념이다.

- **큰 K**: 높은 편향이지만 낮은 분산. 클래스 불균형 문제
- **최적화 방법**: 교차 검증(cross-validation)을 통한 K 선택
- **공간 분할 방법**: KD-트리, R-트리 등의 인덱스 구조 사용
- **근사 기법**: 정확한 K-NN 대신 근사 근접 이웃 검색(ANN) 활용
- **차원 축소**: PCA 등으로 데이터 차원 감소
- 고차원 데이터에서 "차원의 저주" 문제
- K값 선택의 민감성
- **고차원 벡터 공간에서의 거리 메트릭 선택**: 현대의 임베딩 모델(예: BERT, GPT)에서 생성되는 고차원 벡터들에 대해 어떤 거리 메트릭이 가장 적절한가?

- **필터 조건과 K-NN의 상호작용:** 속성 기반 필터를 적용할 때 정확한 K개의 결과를 보장하기 위한 최적 전략은 무엇인가?
- **동적 K값 조정:** Exquator가 동적으로 K값을 조정하여 필터링된 결과의 질을 개선할 수 있을까?
- **거리 계산의 정규화:** 벡터 정규화가 필터링 성능에 미치는 영향에 대한 실증적 분석이 필요하다.

[36] A review of various k-nearest neighbor query processing techniques

KNN 쿼리는 데이터베이스에서 가장 흔하면서도 계산 비용이 높은 작업 중 하나다:

문제 정의:

- 주어진 쿼리 포인트 q 와 정수 K 에 대해
- 거리 $d(q, p)$ 기준으로 가장 가까운 K 개의 포인트 p 를 찾기
- 범위 쿼리보다 복잡: 사전에 결과 집합의 크기를 알 수 없음

도전 과제:

- 전체 데이터와 거리 계산 필요 (순차 탐색: $O(n)$)
- 고차원에서 인덱스 구조의 효율성 저하
- 원본 데이터 공간에서의 복잡한 거리 계산
- 고차원에서 성능 저하 문제 ("차원의 저주")

R-트리(R-Tree)

- 다차원 공간의 위치 정보를 인덱싱하는 B-트리 기반 구조
- 축 정렬 최소 경계 사각형(MBR) 사용
- 동적 삽입/삭제 지원
- 공간 쿼리 처리에 효율적

R*-트리 및 변형

[37] Generalized k-nearest neighbor rules

- 클래스 불균형 문제 완화

신뢰도 정보:

- 분류 결과의 신뢰도를 정량화
- 낮은 신뢰도의 분류 결과 식별 가능
- 거부 옵션(rejection option) 제공 가능

유연한 해석:

- 분류 결과를 이진 결정 대신 연속적 확률로 표현
- 어플리케이션의 요구에 따른 임계값 조정 가능
- **벡터 공간에서의 퍼지 KNN:** 고차원 벡터 임베딩 공간에서 퍼지 멤버십 함수를 어떻게 정의할 것인가?
- **필터와 거리의 퍼지 결합:** 속성 필터 조건과 벡터 거리를 퍼지 논리로 결합했을 때의 성능 이득은?
- **멤버십 함수 학습:** 데이터로부터 최적의 멤버십 함수를 자동으로 학습할 수 있을까?
- **다중 필터의 퍼지 논리 결합:** 여러 속성 필터의 AND/OR 조합을 퍼지 집합으로 어떻게 표현할까?
- **해석 가능성:** 퍼지 KNN의 결과를 사용자에게 어떻게 해석 가능한 형태로 제시할 것인가?
- **성능과 불확실성:** 퍼지 멤버십 함수가 실제 불확실성을 올바르게 반영하는가?

[38] Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs

알고리즘의 핵심은 계층적 그래프 구조로, 하단의 밀집 그래프에서 상단의 희소 그래프로 이동하면서 검색 공간을 점진적으로 좁혀나간다. 이는 "네비게이팅 작은 세계" 개념(small-world phenomenon)을 그래프에 적용한 것으로, $O(\log n)$ 시간 복잡도의 검색을 실현한다.

본 논문과의 관계: Exqutor 시스템은 HNSW를 기본 인덱싱 구조로 사용하며, 필터링 기반의 조건부 근접 이웃 검색을 HNSW 위에서 구현한다. HNSW의 계층적 구조와 네비게이팅 메커니즘을 활용하여 속성 필터를 함께 고려하는 효율적인 검색을 수행한다.

근접 이웃 검색의 도전:

- 고차원 벡터 공간에서 정확한 KNN 검색: $O(n)$ 선형 탐색 필요
- 대규모 데이터셋(수백만~수십억): 실시간 검색 불가능
- 차원의 저주: 트리 기반 인덱스의 성능 급격히 저하
- 정확도와 속도의 트레이드오프 필요

HNSW의 목표:

- 근사 최근접 이웃만 검색하되 매우 빠르고 정확
- $O(\log n)$ 시간 복잡도 달성
- 메모리 효율적인 인덱스 구조
- 높은 차원(1000+)에서도 안정적 성능

계층 개념:

- 다중 계층(multiple layers)으로 이루어진 그래프 구조
- 각 계층은 별도의 그래프로 표현

[39] Fast approximate nearest neighbor search with the navigating spreading-out graph

본 논문은 HNSW와 경쟁하는 또 다른 강력한 그래프 기반 ANN 인덱싱 기법인 NSG(Navigating Spreading-out Graph, 네비게이팅 스프레딩아웃 그래프)를 제시한다. NSG는 단일 계층의 그래프 구조를 사용하면서도 HNSW와 유사한 검색 성능을 달성하고, 메모리 효율성과 구성 속도 면에서 우위를 가진다.

NSG의 핵심은 "spreading-out" 개념으로, 각 노드가 다양한 방향의 이웃들을 균형 있게 연결하도록 한다. 이는 그래프의 네비게이팅 능력(navigability)을 높이면서도 불필요한 간선을 줄인다.

본 논문과의 관계: Exqutor가 HNSW 대신 NSG를 사용할 경우의 성능 특성을 이해하는 데 중요하다. 단일 계층 구조가 필터링과 결합될 때 메모리 효율성과 검색 정확도 간의 트레이드오프를 분석할 수 있다.

- 다중 계층 구조 (hierarchical)
- 상단부터 하단으로 네비게이팅
- 삽입 시간 많이 소요
- 높은 메모리 사용량

NSG (Navigating Spreading-out):

- 단일 계층 구조 (flat)
- 직접 그래프 구성
- 더 빠른 구성 시간

- 더 낮은 메모리 오버헤드
- 검색 성능은 HNSW와 경쟁 가능
- 그래프 구조 최적화로 어디서 시작해도 비슷한 성능

시작점 선택 알고리즘:

[40] Song: Approximate nearest neighbor search on gpu

SONG의 핵심은 공간 정렬(spatial ordering)을 통해 벡터들을 구조화하고, GPU 메모리 계층을 고려한 최적화된 네비게이팅 그래프를 구성하는 것이다. 배치 쿼리 처리와 대규모 병렬화로 CPU 기반 방식보다 훨씬 빠른 성능을 달성한다.

본 논문과의 관계: Exqutor가 필터링 기반 벡터 검색을 GPU에서 구현할 경우, SONG의 공간 정렬과 GPU 최적화 기법을 활용할 수 있다. 특히 대규모 쿼리 배치 처리 시 필터링과 거리 계산을 GPU에서 병렬로 수행하는 전략이 효과적이다.

GPU 메모리 구조:

```

레지스터 (Register)
  ↓ (매우 빠름, 제한적)
공유 메모리 (Shared Memory)
  ↓ (빠름, 블록 내 공유)
글로벌 메모리 (Global Memory)
  ↓ (느림, 높은 지연)
메인 메모리 (Host Memory)
  
```

최적화 전략:

1. **데이터 지역성:** 자주 접근되는 데이터를 공유 메모리에 캐싱
2. **메모리 접근 패턴:** 연속적 메모리 접근으로 높은 대역폭 활용

[41] Filtered-DiskANN: Graph algorithms for approximate nearest neighbor search with filters

본 논문과의 관계: Exqutor의 핵심 문제인 "필터 조건과 거리 기반 검색의 효율적 결합"을 직접 다루는 논문이다. Filtered-DiskANN의 설계 원칙과 최적화 기법은 Exqutor의 필터링 전략과 매우 유사하다.

- 메모리에는 최소한의 구조만 유지
- Vamana 그래프 알고리즘 기반

메모리-디스크 계층:

빠르고 작음	←	느리고 큼
메모리 (GPU/RAM)	←	디스크 (SSD)
↑		↑
핫 데이터		콜드 데이터
(최근 접근)		(대부분)

DiskANN의 장점:

- 메모리 제약 극복: 수십억 개 벡터 지원
- 비용 효율적: SSD 가격이 RAM보다 저렴
- 높은 성능: 벡터당 수 밀리초 검색

[42] PM-LSH; A fast and accurate lsh framework for high-dimensional approximate nearest neighbor search

레이어 구조:

```

layer 1:  $h_1 = [h^{(1)}_1, h^{(1)}_2, \dots, h^{(1)}_k] \rightarrow$  해시값 1
           (k개 함수의 연결)
layer 2:  $h_2 = [h^{(2)}_1, h^{(2)}_2, \dots, h^{(2)}_k] \rightarrow$  해시값 2
           ...
layer L:  $h_L = [h^{(L)}_1, h^{(L)}_2, \dots, h^{(L)}_k] \rightarrow$  해시값 L
  
```

검색 프로세스:

1. 쿼리 q 의 각 레이어별 해시값 계산
 2. 각 레이어에서 같은 해시 버킷의 모든 점 찾기
 3. 찾은 모든 점들 중 가장 가까운 K 개 선택
- 시간복잡도: $O(L \times \text{hash_computation} + \text{후보_수})$

문제점 분석:

[43] Neighbor-sensitive hashing

본 논문은 Neighbor-Sensitive Hashing(NSH)라는 새로운 해싱 기반 ANN 기법을 제시한다. 전통적인 LSH와 달리, NSH는 각 데이터 포인트의 이웃 구조를 명시적으로 활용하여 해시 함수를 설계한다.

NSH의 핵심 아이디어는 "이웃들이 같은 해시 버킷에 들어갈 확률을 높인다"는 것이다. 이를 위해 실제 데이터로부터 학습된 이웃 그래프를 기반으로 최적화된 해시 함수를 구성한다.

본 논문과의 관계: Exqutor의 필터링 기반 검색에서 필터 조건을 만족하는 벡터들의 이웃 구조를 활용하면, NSH처럼 필터 친화적인 인덱싱을 구성할 수 있다.

LSH의 문제점:

LSH는 무작위 함수를 사용:

- 데이터 분포 미고려
- 실제 이웃 관계 무시
- 높은 거짓양성율

문제:

- 벡터 A와 B가 유사하지만
- LSH 함수의 무작위성으로 인해
- 다른 해시 버킷에 들어갈 수 있음

NSH의 아이디어:

[44] Scalable nearest neighbor algorithms for high dimensional data

본 논문은 고차원 데이터에서의 확장 가능한 근접 이웃 검색을 위한 FLANN(Fast Library for Approximate Nearest Neighbors) 라이브러리를 제시한다. FLANN의 핵심 기여는 주어진 데이터셋과 성능 요구사항에 따라 최적의 ANN 알고리즘을 자동으로 선택하는 메커니즘이다.

다양한 ANN 기법(KD-트리, LSH, 랜덤 포레스트 등)을 통합하고, 메타-학습을 통해 데이터 특성에 가장 적합한 알고리즘과 파라미터를 자동으로 결정한다.

본 논문과의 관계: Exqutor가 다양한 필터 조건과 데이터 분포에 대응하기 위해, FLANN의 자동 알고리즘 선택 개념을 적용하면 동적으로 최적의 검색 전략을 선택할 수 있다.

고차원에서의 문제:

트리 기반 인덱스 (KD-트리, R-트리):

- 차원의 저주로 인한 성능 저하
- 높은 차원: $O(n)$ 에 가까운 성능

LSH:

- 파라미터(k , L) 선택 어려움
- 데이터 분포에 따라 성능 큰 변동

HNSW:

- 높은 메모리 요구
- 파라미터 튜닝 필요

→ 실제로는 알고리즘과 파라미터를 올바르게 선택하기 어려움

[45] VHP: Approximate nearest neighbor search via virtual hypersphere partitioning

본 논문과의 관계: Exqutor의 필터링 기반 검색에서 공간 분할 구조를 활용하면, 같은 분할 영역 내의 필터 조건을 만족하는 벡터들을 빠르게 찾을 수 있다.

→ 동심원 구조 (concentric spheres)

예시:

중심: c

초구 1: 반지름 r_1 내의 점들

초구 2: $r_1 < \text{거리} \leq r_2$ 영역의 점들

초구 3: $r_2 < \text{거리} \leq r_3$ 영역의 점들

...

장점:

- 중심에 가까운 점일수록 더 세밀한 분할
- 각도 정보와 거리 정보 동시 활용

****계층적 VHP 구성:****

루트 중심: 전체 데이터의 중심

[46] Annoy: Approximate Nearest Neighbors in C++/Python

Annoy의 주요 특징은 메모리 효율성과 빠른 쿼리 속도이다. Random projection tree 구조를 이용하여 고차원 공간을 재귀적으로 분할하고, 이진 파일 형태로 인덱스를 저장할 수 있다. 메모리 매핑을 지원하므로 매우 큰 데이터셋도 효율적으로 처리할 수 있으며, 실시간 검색 애플리케이션에서 광범위하게 사용되었다.

Annoy의 Random Projection Trees는 고차원 벡터를 임의로 선택된 두 점을 지나는 하이퍼플레인으로 분할하는 방식으로 작동한다. 이 접근방식은 구현이 간단하면서도 효과적이며, 실무 환경에서 높은 성능을 보여준다. 검색 과정에서는 리프 노드 근처의 후보들을 탐색하고, 필요시 추가 트리를 검사하여 정확도를 조정할 수 있다.

메모리 효율성 측면에서 Annoy는 각 벡터를 부동소수점으로 저장하며, 인덱스 구조도 매우 컴팩트하다. 이는 모바일 기기나 엣지 디바이스에서도 대규모 벡터 데이터를 처리할 수 있게 한다. 특히 음악 추천 시스템에서 백만 개 이상의 곡을 실시간으로 검색해야 하는 Spotify의 요구사항을 충족하기 위해 설계되었다.

Exqutor와의 관계를 살펴보면, Annoy는 유사성 검색 인덱싱 기술의 발전을 보여주는 중요한 작업이다. Exqutor는 유사성 쿼리의 카디널리티 추정을 다루는데, 이는 ANN 검색 결과에 기반한 시스템에서 쿼리 최적화를 위해 중요하다. 따라서 Annoy와 같은 효율적인 ANN 구조를 이해하는 것은 Exqutor의 카디널리티 추정 모델이 실제 검색 시스템에서 어떻게 적용되는지 이해하는 데 필수적이다.

Random Projection Trees (RP-Trees)는 Annoy의 핵심 자료구조이다. 이 구조는 고차원 벡터 공간을 재귀적으로 분할하는 방식으로 작동한다. 구체적으로, 각 내부 노드에서는 임의로 선택된 두 점을 지나가는 하이퍼플레인을 사용하여 벡터들을 두 그룹으로 나눈다.

하이퍼플레인의 선택 방식은 다음과 같다. 현재 노드에 할당된 벡터 집합에서 임의로 두 점 p_1 과 p_2 를 선택하고, 이 두 점의 중점을 지나면서 $p_2 - p_1$ 벡터에 수직인 하이퍼플레인을 정의한다. 이 하이퍼플레인을 기준으로 각 벡터를 분류하여 왼쪽 또는 오른쪽 자식 노드로 할당한다.

이러한 방식의 장점은 계산의 간단성과 효율성에 있다. 하이퍼플레인 정의에 필요한 계산량이 적으므로, 대규모 데이터셋에 대해서도 빠르게 인덱스를 구축할 수 있다. 또한 임의성으로 인해 자동적으로 어느 정도 로드 밸런싱이 이루어지므로, 트리가 한쪽으로 치우치지 않도록 한다.

리프 노드에 도달하면 해당 노드의 모든 벡터들이 저장된다. 리프 노드의 크기는 하이퍼파라미터로 조정할 수 있으며, 이는 메모리 사용량과 쿼리 속도 간의 트레이드오프를 제어한다.

Annoy의 또 다른 핵심 특징은 메모리 효율성이다. 인덱스를 바이너리 파일로 저장할 때, 각 벡터는 실수 배열로 저장되며, 트리 구조는 매우 간결하게 인코딩된다. 일반적으로 각 차원당 4바이트(float32) 또는 8바이트(float64)만 사용한다.

메모리 매핑(Memory Mapping) 기능을 지원하므로, 디스크의 인덱스 파일을 직접 메모리에 매핑하여 사용할 수 있다. 이는 매우 큰 데이터셋의 경우 메인 메모리의 크기에 제약받지 않고도 검색을 수행할 수 있다는 의미이다.

인덱스 파일의 구조는 헤더 정보(벡터 차원수, 트리 개수 등)와 인덱스 데이터로 구성된다. 트리 구조는 배열 기반의 이진 트리 표현을 사용하여, 포인터 오버헤드를 최소화한다.

- 1. 정확도 보장의 부재:** Annoy는 근사 알고리즘이므로 항상 실제 최근접 이웃을 찾는다라는 보장이 없다. 애플리케이션에서 이러한 근사성이 허용 가능한 수준인지 검증이 필요하다.
- 2. 벡터 추가의 어려움:** Annoy는 정적 인덱스이므로, 새로운 벡터를 추가하려면 전체 인덱스를 재구축해야 한다. 동적 데이터셋에는 부적합할 수 있다.
- 3. 고차원에서의 성능 저하:** Random projection은 고차원에서 curse of dimensionality를 완전히 극복하지는 못한다. 벡터의 차원이 매우 높으면 성능이 저하될 수 있다.
- 4. 카디널리티 추정과의 통합:** Annoy 같은 ANN 인덱스를 사용하는 시스템에서, 유사성 검색의 결과 크기를 정확하게 예측하려면 별도의 카디널리티 추정 기법이 필요하다. 이것이 Exqutor가 다루는 핵심 문제이다.

[47] Product Quantization for Nearest Neighbor Search

Product Quantization은 고차원 벡터의 유사성 검색 문제를 효율적으로 해결하기 위해 설계된 기법이다. 기본 아이디어는 'product'라는 이름에서도 알 수 있듯이, 전체 공간을 여러 개의 저차원 부분공간의 곱(product)으로 분해하는 것이다.

구체적인 프로세스는 다음과 같다. 먼저, d 차원 벡터를 m 개의 d/m 차원 부분벡터로 분할한다. 예를 들어 $d=128$, $m=4$ 인 경우, 각 부분벡터는 32차원이다. 각 부분공간에 대해 독립적인 양자화(quantization)를 수행한다. 이는 k -means 클러스터링을 사용하여 각 부분공간에서 k 개의 클러스터 중심(양자화 중심)을 학습하는 것을 의미한다.

양자화 중심의 개수 k 는 각 부분벡터를 몇 비트로 인코딩할 것인지에 따라 결정된다. 일반적으로 $k=256$ (8비트)을 사용하므로, 전체 벡터는 m 개의 8비트 코드의 조합, 즉 8m비트로 표현된다. 128차원을 8비트로 압축하면 약 16배 압축이 되며, 원본 벡터 ($128 \times 4\text{바이트} = 512\text{바이트}$) 대비 4바이트만으로 표현할 수 있다.

각 부분공간에서의 양자화는 다음과 같이 진행된다. 부분공간 j 에 속한 모든 벡터들의 부분벡터들을 추출하고, 이에 대해 k -means 알고리즘을 실행한다. 결과적으로 k 개의 중심 벡터 $c_{j,0}, c_{j,1}, \dots, c_{j,k-1}$ 이 학습된다. 각 훈련 벡터의 부분벡터는 가장 가까운 중심에 할당되고, 그 중심의 인덱스를 코드로 저장한다.

PQ는 손실 압축(lossy compression)이므로, 당연히 정보 손실이 발생한다. 그러나 유사성 검색의 맥락에서는 이러한 손실이 심각한 문제가 되지 않을 수 있다. 왜냐하면 우리가 관심 있는 것은 정확한 거리 값이 아니라 상대적인 거리 순서이기 때문이다.

PQ의 정확도는 여러 요인에 의해 영향을 받는다. 첫째, 부분공간의 개수 m 이다. m 이 작을수록 압축률은 높지만 정확도는 낮다. 일반적으로 $m=4\sim 8$ 정도가 좋은 균형을 제공한다.

둘째, 각 부분공간의 양자화 중심 개수 k 이다. k 가 작을수록 압축률은 높지만 정확도는 낮다. $k=256$ (8비트)은 많은 응용에서 충분히 높은 정확도를 제공한다.

셋째, 학습 데이터와 쿼리 데이터의 분포의 일치도이다. PQ 모델이 학습한 분포와 실제 쿼리 분포가 다르면 정확도가 저하될 수 있다.

실제 실험에서 PQ는 다음과 같은 특성을 보여준다. SIFT 1M 데이터셋에서 99%의 상대적 정확도(recall@1000)를 달성하면서 압축률은 약 32배를 제공한다. 또한 거리 계산 속도는 원본 벡터 사용 시보다 수십 배 빠르다.

- 부분공간 분할의 최적화:** 부분공간을 독립적으로 분할할 때, 최적의 분할 방식에 대한 이론적 지침이 부족하다. 데이터 특성에 따라 최적의 m 값을 선택하는 방법이 명확하지 않다.
- 동적 데이터에의 적용:** PQ 모델이 학습된 후 새로운 벡터가 추가될 때, 모델을 재학습해야 하는지 아니면 기존 중심을 사용할 수 있는지에 대한 명확한 지침이 없다.
- 카디널리티 추정의 어려움:** PQ 압축으로 인한 거리 변형이 유사성 쿼리의 결과 크기 예측에 미치는 영향을 정량화하기 어렵다. 이것이 Exqutor 같은 학습 기반 추정 방법이 필요한 이유이다.
- 고차원의 한계:** 벡터의 차원이 매우 높아지면, 부분공간도 많아져야 하고, 이에 따라 계산 오버헤드가 증가한다.

[48] Faiss: A Library for Efficient Similarity Search and Clustering of Dense Vectors

또 다른 중요한 방법은 HNSW (Hierarchical Navigable Small World) 기반의 인덱싱이다. HNSW는 그래프 기반의 방법으로, 인접한 벡터들을 연결하는 계층 구조를 만든다. 검색은 상위 계층에서 시작하여 하위 계층으로 내려가면서 진행되며, 매우 빠른 검색 속도를 제공한다.

또 다른 구성은 다층 인덱싱(Multi-level Indexing)이다. 예를 들어, 먼저 LSH를 사용하여 대략적으로 후보를 선택하고, 그 다음 IVF를 사용하여 더 정밀하게 검색한다. 이러한 다층 구조는 정확도와 성능의 균형을 더 잘 맞출 수 있다.

Faiss는 단순히 유사성 검색만 제공하는 것이 아니라, 벡터 클러스터링 기능도 제공한다. 이는 데이터의 구조를 이해하고 그룹화하는 데 유용하다.

k-means 클러스터링은 Faiss의 핵심 기능 중 하나이다. 일반적인 k-means 구현과 달리, Faiss의 k-means는 GPU 가속을 지원하므로 매우 빠르다. 수억 개의 벡터를 수십 초 내에 클러스터링할 수 있다.

클러스터링의 결과는 다양하게 활용될 수 있다. 예를 들어, 클러스터 중심을 사용하여 IVF 인덱싱의 클러스터를 초기화할 수 있다. 또한 클러스터링 결과 자체를 분석하여 데이터의 특성을 이해할 수 있다.

Exqutor는 유사성 쿼리의 카디널리티 추정을 다룬다. Faiss는 벡터 검색의 가장 중요한 인프라로, Exqutor가 다루는 추정 문제는 Faiss와 같은 벡터 검색 시스템에서 직접 발생한다.

특히, Faiss의 다양한 인덱싱 방법(IVF, PQ, HNSW 등)들은 검색 결과의 크기를 예측하기 어렵게 만든다. 예를 들어, 특정 거리 임계값 이하의 벡터를 검색할 때, 정확히 몇 개의 벡터가 반환될지 예측하는 것은 매우 어렵다. Exqutor의 머신러닝 기반 카디널리티 추정 기법은 Faiss와 같은 현대적인 벡터 검색 시스템에서 정확한 카디널리티 추정을 가능하게 한다.

또한 Faiss의 GPU 가속이 가능해졌다는 사실은 대규모 벡터 검색에서 카디널리티 추정의 중요성을 더욱 강조한다. GPU를 활용한 검색이 매우 빨라지면서, 쿼리 최적화의 오버헤드가 상대적으로 더 중요해진다. 따라서 정확한 카디널리티 추정으로 최적의 실행 계획을 선택하는 것이 전체 성능에 미치는 영향이 더욱 커진다.

-
1. **동적 인덱스 업데이트:** Faiss의 많은 인덱싱 방법들은 정적 데이터셋을 가정한다. 새로운 벡터가 지속적으로 추가되는 동적 환경에서의 효율적인 인덱스 관리 방법이 제한적이다.
 2. **카디널리티 추정의 어려움:** Faiss의 근사 검색 방법들은 검색 결과의 크기를 정확하게 예측하기 어렵게 만든다. IVF+PQ 조합의 경우, 각 단계에서의 근사성이 누적되어 최종 결과 크기 예측이 복잡해진다.
 3. **하이퍼파라미터 튜닝:** Faiss는 많은 하이퍼파라미터를 가지고 있으며, 최적의 성능을 위해서는 신중한 튜닝이 필요하다. 특정 데이터셋과 쿼리 패턴에 대한 최적의 설정을 자동으로 결정하는 방법이 부족하다.
 4. **메모리 대역폭의 한계:** GPU 가속의 이점에도 불구하고, 메모리 대역폭(memory bandwidth)은 여전히 대규모 검색의 병목이 될 수 있다.

[49] Billion-Scale Similarity Search with GPUs

논문은 단순히 GPU 가속만을 다루는 것이 아니라, billion-scale에서 발생하는 다양한 실무적 문제들을 해결하는 방법을 제시한다. 메모리 관리, 배치 처리, 여러 GPU의 효율적 활용, 네트워크 통신 최적화 등이 포함된다. 특히 메모리 제약을 극복하기 위해 PQ (Product Quantization)와 같은 압축 기법을 GPU 레벨에서 효율적으로 구현하는 방법을 보여준다.

GPU 메모리의 계층 구조도 중요하다. GPU는 레지스터(register), 공유 메모리(shared memory), 글로벌 메모리(global memory) 등 여러 단계의 메모리를 가지고 있다. 가장 빠른 성능을 위해서는 접근 패턴(access pattern)을 신중하게 설계하여, 자주 접근하는 데이터를 빠른 메모리에 유지해야 한다.

Coalesced memory access는 GPU 성능의 핵심이다. 여러 스레드가 메모리에서 연속된 주소를 접근할 때 (coalesced access), 한 번의 메모리 트랜잭션으로 여러 데이터를 가져올 수 있다. 따라서 벡터 데이터의 메모리 레이아웃을 신중하게 설계하는 것이 중요하다.

CPU의 주 메모리(main memory)가 수백 GB에서 TB 단위인 반면, GPU의 메모리는 일반적으로 4GB에서 40GB 정도이다. 따라서 billion-scale의 벡터 데이터를 GPU에 모두 저장할 수 없다. 이 문제를 해결하기 위해 논문은 다음과 같은 전략들을 제시한다.

첫째, 벡터 압축이다. PQ (Product Quantization)를 사용하면 128차원 벡터를 4바이트로 압축할 수 있다. 따라서 1억 개의 벡터는 400MB에 저장되며, 10억 개도 4GB에 저장할 수 있다. 하지만 이는 정확도 손실을 초래한다.

둘째, 계층화된 인덱싱이다. 먼저 대규모 데이터를 여러 개의 샤드(shard)로 분할한다. 각 샤드는 개별적으로 GPU 메모리에 적재되거나 디스크에 보관된다. 검색 시에는 필요한 샤드만 GPU로 로드하여 검색을 수행하고, 결과를 병합한다.

셋째, 스트리밍 처리(streaming)이다. 벡터 데이터를 한 번에 모두 메모리에 로드하지 않고, 배치 단위로 순차적으로 처리한다. 이를 통해 메모리 제약을 극복할 수 있다.

메모리 대역폭(memory bandwidth)도 중요한 병목이다. GPU의 메모리 대역폭은 CPU보다 높지만, 여전히 billion-scale의 데이터를 빠르게 처리하기에는 제한적일 수 있다. 따라서 메모리 접근을 최소화하는 알고리즘 설계가 중요하다.

Exqutor는 유사성 쿼리의 카디널리티 추정을 다룬다. 본 논문이 다루는 GPU 기반 billion-scale 검색은 Exqutor가 다루는 카디널리티 추정 문제를 더욱 시급하게 만든다.

GPU를 사용한 검색이 매우 빨라지면서, 쿼리 최적화의 중요성이 상대적으로 더 커진다. 효율적인 실행 계획을 선택하기 위해서는 정확한 카디널리티 추정이 필수적이다. 특히 billion-scale에서는 몇 프로센트의 성능 차이도 초당 수백 쿼리를 처리하는 시스템에서는 무시할 수 없는 차이가 된다.

또한 IVF+PQ의 근사 검색은 결과 크기의 예측을 어렵게 만든다. IVF의 클러스터 선택에서의 근사성과 PQ의 거리 변형이 누적되어, 최종 결과 크기를 정확하게 예측하기 어렵다. Exqutor의 머신러닝 기반 추정 기법은 이러한 복잡한 시스템에서도 정확한 카디널리티 추정을 제공한다.

- 메모리 대역폭의 한계:** GPU 메모리 대역폭은 높지만, billion-scale 데이터에서는 여전히 병목이 될 수 있다. 메모리 대역폭을 더욱 효율적으로 활용하는 방법이 필요하다.
- 정확도와 속도의 트레이드오프:** GPU 가속은 속도를 극적으로 높이지만, 검색 정확도와와의 트레이드오프가 존재한다. PQ 압축 수준을 높이면 메모리와 속도는 이점을 얻지만 정확도가 낮아진다.
- 동적 업데이트의 어려움:** billion-scale 데이터셋에서 새로운 벡터를 추가하거나 기존 벡터를 삭제하는 것은 복잡한 작업이다. 인덱스를 효율적으로 유지하면서 동시성을 제어하는 방법이 필요하다.

[50] Exact Cardinality Query Optimization with Bounded Execution Cost

전통적인 쿼리 최적화는 주로 통계 정보를 기반으로 카디널리티를 추정한다. 하지만 이러한 추정은 종종 부정확하며, 특히 유사성 검색과 같은 복잡한 쿼리에서는 추정 오류가 심각할 수 있다. Trummer는 이 문제를 해결하기 위해, 제한된 실행 비용(bounded execution cost) 내에서 정확한 카디널리티를 결정할 수 있는 방법을 제시했다.

ECQO의 핵심 아이디어는 다음과 같다. 완전한 쿼리 실행 대신, 부분적 실행을 통해 결과의 크기를 정확히 파악할 수 있다면, 이를 기반으로 최적의 실행 계획을 선택할 수 있다는 것이다. 예를 들어, 조인(join) 쿼리에서 각 조인 단계 후의 정확한 카디널리티를 알면, 다음 조인의 순서를 동적으로 최적화할 수 있다.

이 접근법은 특히 유사성 검색과 같은 새로운 쿼리 유형에 매우 적합하다. 유사성 쿼리의 결과 크기는 거리 임계값에 매우 민감하며, 이를 정확하게 예측하는 것은 매우 어렵다. 그러나 실제 데이터에 대해 부분적으로 쿼리를 실행해보면, 정확한 결과 크기를 빠르게 파악할 수 있다.

본 논문은 Exqutor의 직접적인 영감이 되었으며, Exqutor는 이 개념을 한 단계 더 발전시켜 머신러닝을 사용한 카디널리티 추정 방법을 제시한다. ECQO는 원리적 접근(principled approach)을 제시했고, Exqutor는 실무적 구현을 제공한 것이다.

전통적인 쿼리 최적화에서 카디널리티 추정은 매우 중요한 역할을 한다. 쿼리 옵티마이저는 추정된 카디널리티를 기반으로 실행 계획을 선택하기 때문이다. 예를 들어, 조인 쿼리에서 A 테이블과 B 테이블을 조인할 때, A를 먼저 필터링한 후 B와 조인하는 것이 좋은지, 아니면 B를 먼저 필터링하는 것이 좋은지는 각 단계 후의 카디널리티에 따라 결정된다.

전통적인 카디널리티 추정은 다음과 같은 문제점을 가진다. 첫째, 통계 정보에 기반하고 있다. 이는 실제 데이터 분포와 다를 수 있다. 예를 들어, 테이블의 히스토그램(histogram)이 실제 값의 분포를 완벽하게 반영하지 못할 수 있다.

둘째, 독립성 가정(independence assumption)을 기반으로 한다. 여러 조건이 결합될 때, 일반적으로 각 조건이 독립적이라고 가정하고 확률을 곱한다. 하지만 현실에서 여러 조건은 종종 상관관계가 있다. 예를 들어, 도시(city) 필터와 우편번호(zip code) 필터가 결합되었을 때, 이 두 조건은 독립적이지 않으므로 확률 곱셈은 부정확하다.

셋째, 복잡한 쿼리에서는 추정 오류가 누적된다. 여러 조인 단계를 거치면서, 각 단계의 추정 오류가 누적되어 최종 추정값의 정확도가 심각하게 악화될 수 있다.

넷째, 새로운 쿼리 유형(예: 유사성 검색, 그래프 쿼리)에 대해서는 통계 기반의 추정 방법이 적용되기 어렵다. 벡터 거리의 분포나 유사성 쿼리의 결과 크기를 전통적인 통계 방법으로 추정하는 것은 매우 어렵다.

세 번째 방법은 근사 구조(approximation structures)를 사용하는 것이다. 예를 들어, 블룸 필터(Bloom filter)나 count-min sketch 같은 확률적 자료구조를 사용하면, 최소한의 메모리와 계산으로도 근사적인 결과 개수를 빠르게 얻을 수 있다. 이를 정확한 개수로 수렴하도록 반복적으로 개선할 수 있다.

Exqutor는 이러한 문제를 다르게 접근한다:

- 머신러닝 모델을 사용하여 쿼리 특성으로부터 직접 카디널리티를 예측한다
- 모델 학습은 사전에 수행되므로, 런타임 오버헤드가 매우 적다
- 부분 실행 대신, 벡터 검색의 특성(벡터 차원, 거리 임계값, 인덱스 구조 등)을 특징으로 사용한다

특히 유사성 쿼리에서 Exqutor는 다음과 같은 추가적인 고려사항을 반영한다:

[51] Learned Cardinality Estimation for Similarity Queries (SelNet)

SelNet의 핵심 아이디어는 단순하면서도 강력하다: 유사성 쿼리의 선택도(selectivity)를 예측하는 문제를 회귀 문제(regression problem)로 바라보고, 신경망을 사용하여 이를 학습하는 것이다. 선택도는 쿼리 결과의 개수를 데이터셋 전체 크기로 나눈 비율이다. 예를 들어, 100만 개의 벡터 중에서 1000개가 검색 조건을 만족한다면, 선택도는 0.001이다. SelNet은 다음과 같은 특징을 갖는다. 첫째, 유사성 쿼리의 특성(벡터 차원, 거리 임계값, 쿼리 벡터의 특성 등)을 입력으로 받아, 선택도를 직접 예측한다. 둘째, 신경망 기반의 접근은 벡터 공간의 복잡한 특성을 자동으로 학습할 수 있다. 셋째, 다양한 데이터셋과 쿼리 패턴에 대해 충분한 학습 데이터가 있다면, 높은 정확도를 달성할 수 있다. SelNet의 성능은 매우 인상적이다. 전통적인 통계 기반 추정 방법과 비교했을 때, 추정 오류를 평균 80% 이상 감소시킨다. 즉, 전통적 방법의 추정 오류가 5배라면, SelNet은 1배 이내의 오류로 줄일 수 있다는 뜻이다. 하지만 SelNet도 한계가 있다. 학습 데이터에 포함되지 않은 새로운 패턴의 쿼리에 대해서는 성능이 저하될 수 있다 (분포 이동, distribution shift). 또한 신경망의 구조와 하이퍼파라미터 선택에 따라 성능이 크게 달라질 수 있다. Exqutor는 이러한 SelNet의 한계를 개선하는 것을 목표로 한다.

넷째, 새로운 인덱스 구조에의 적응이다. 만약 IVF 인덱싱을 사용하다가 HNSW로 변경하면, 모델의 성능이 저하될 수 있다. 인덱스 구조도 입력 특징에 포함되어야 하며, 각 인덱스 구조에 대해 별도의 학습이 필요할 수 있다.

SelNet은 유사성 쿼리의 카디널리티 추정에 머신러닝을 처음 적용한 중요한 작업이다. 이는 기존의 통계 기반 추정 방법이 유사성 쿼리에 부적합하다는 것을 보여주었고, 머신러닝이 이 문제를 해결할 수 있는 강력한 도구임을 증명했다.

Exqutor는 SelNet의 핵심 아이디어를 계승하면서도, 다음과 같은 측면에서 개선을 이룬다:

- 더 강력하고 정확한 학습 모델
- 실행 비용 제약을 고려한 최적화
- 부분 실행과 학습 기반 추정의 효과적인 결합
- 더 포괄적인 실험과 평가

1. **모델의 일반화:** SelNet 모델이 학습한 데이터와 다른 분포의 새로운 데이터에 얼마나 잘 일반화되는가? 특히 매우 높은 차원이나 매우 낮은 차원의 데이터에는 어떻게 적응하는가?
2. **온라인 학습과 적응:** 실제 운영 환경에서 지속적으로 들어오는 새로운 쿼리들을 어떻게 활용하여 모델을 개선할 것인가? 모델 재학습의 빈도와 비용은?
3. **인덱스 구조의 영향:** 동일한 데이터에 대해 다양한 인덱스 구조(IVF, HNSW, LSH 등)를 사용할 때, 모델의 성능이 어떻게 달라지는가? 인덱스 구조별로 별도의 모델을 학습해야 하는가?

[52] Kepler: Robust Learning for Parametric Query Optimization

셋째, 샘플된 모든 경우에서 좋은 성능을 제공하는 계획을 찾는다. 이는 다중 목표 최적화(multi-objective optimization) 문제로 볼 수 있다.

1. **매개변수 범위의 정의:** 매개변수의 가능한 범위를 어떻게 정의할 것인가? 실제로는 매개변수가 제약 없이 변할 수 있지만, 학습은 제한된 범위에서만 수행된다.
2. **다중 매개변수:** 여러 매개변수가 결합되었을 때 (예: WHERE col1 > ? AND col2 < ?), 학습 데이터의 차원이 급격히 증가한다 (차원의 저주).
3. **새로운 데이터 패턴:** 데이터가 변해서 이전의 최적 계획이 더 이상 최적이지 아닐 때, 모델을 언제 어떻게 재학습할 것인가?
4. **해석가능성:** 모델이 특정 매개변수 값에 대해 특정 계획을 추천한 이유를 설명할 수 있는가?

[53] Exact Cardinality Query Optimization for Optimizer Testing

이 논문의 주요 기여는 다음과 같다. 첫째, 부분 쿼리 실행을 통해 정확한 카디널리티를 얻을 수 있음을 보여주었다. 둘째, 이를 통해 옵티마이저 테스트 환경에서 실제 최적 계획과 옵티마이저가 선택한 계획을 비교할 수 있음을 제시했다.

본 논문은 Exqutor와 직접적인 관계가 있다. Exqutor는 이 원점 논문의 개념을 계승하면서도, 카디널리티 추정을 더욱 효율적으로 만들기 위해 머신러닝을 도입한다.

데이터베이스 옵티마이저의 성능을 평가하는 것은 복잡한 문제다. 옵티마이저가 선택한 실행 계획의 좋고 나쁨을 판단하려면, 다른 가능한 계획들과 비교해야 한다. 하지만 모든 가능한 실행 계획을 비교하는 것은 계산적으로 불가능하다.

전통적인 벤치마킹 방식은 표준 워크로드(예: TPC-H)를 사용하여, 여러 옵티마이저의 성능을 비교하는 것이다. 그러나 이 방식에는 근본적인 문제가 있다.

첫째, 옵티마이저가 생성한 계획이 느린 경우, 그 원인이 옵티마이저의 부실한 판단인지, 아니면 카디널리티 추정의 부정확함 때문인지 구분하기 어렵다.

둘째, 만약 카디널리티 추정이 완벽하다면 옵티마이저가 정말 최적의 계획을 선택했을 텐데, 이를 알 수 없다.

셋째, 특정 쿼리 패턴에서 옵티마이저의 약점을 파악하기 어렵다.

넷째, 확률적 자료구조를 사용한 추정이다. 블룸 필터나 카운트-민 스케치 같은 자료구조를 사용하면, 매우 적은 메모리로 근사적 카디널리티를 빠르게 계산할 수 있다.

[53] 논문 (Chaudhuri et al. 2009)은 ECQO의 원래 개념을 제시했다. [50] 논문 (Trummer 2019)은 이를 일반적인 쿼리 최적화 문제로 확대했고, Exqutor는 이를 유사성 쿼리의 카디널리티 추정이라는 구체적인 문제로 특화시켰다.

논문들의 진화:

- [53] (2009): 옵티마이저 테스트를 위한 부분 실행 기반 정확한 카디널리티
- [50] (2019): 부분 실행을 통한 일반적인 쿼리 최적화 (ECQO)
- Exqutor: 머신러닝을 사용한 유사성 쿼리의 카디널리티 추정

각 단계별로 다음과 같은 발전이 있었다:

1. 개념 제시: 정확한 카디널리티의 가치 (Chaudhuri et al.)

[54] Analyzing Query Optimizer Performance in the Presence and Absence of Cardinality Estimates

첫째, 카디널리티 추정이 중요함을 보여준다. 카디널리티 추정은 단순히 통계적 흥미로운 문제가 아니라, 실제 데이터베이스 성능에 극적인 영향을 미치는 중요한 문제다.

둘째, 기존 방법의 한계를 명확히 한다. 전통적인 통계 기반 추정의 평균 10배 오류는 시스템 수준에서 용인 불가능한 수준이다.

셋째, 새로운 접근 방식의 필요성을 강조한다. 머신러닝 기반의 카디널리티 추정 (예: SelNet, Exqutor)이 이러한 문제를 해결할 수 있음을 시사한다.

Exqutor의 맥락에서 보면, 이 논문의 분석은 유사성 쿼리의 카디널리티 추정이 얼마나 중요한지를 강조한다. 벡터 데이터베이스 시스템도 유사한 문제를 가지고 있을 것이고, Exqutor의 머신러닝 기반 접근이 이를 해결할 수 있음을 시사한다.

1. **적응형 쿼리 실행:** 실행 중에 카디널리티 추정을 수정하고 계획을 변경할 수 있는 적응형(adaptive) 쿼리 처리의 이점은?
2. **통계 정확도의 유지:** 데이터가 지속적으로 변하는 환경에서, 통계를 정확하게 유지하는 방법은?
3. **학습 기반 방법의 효과:** 이 논문의 발견이 머신러닝 기반 카디널리티 추정의 필요성을 얼마나 강력하게 지지하는가?
4. **종합적 해결책:** 카디널리티 추정만으로 충분인가, 아니면 비용 모델(cost model)의 개선도 동시에 필요한가?

[55] Analyzing the Impact of Cardinality Estimation on Execution Plans in Microsoft SQL Server

논문의 핵심 기여는 다음과 같다. 첫째, SQL Server 특화의 카디널리티 추정 오류 패턴을 식별한다. 둘째, 이러한 오류가 실행 계획 선택에 미치는 구체적인 영향을 정량화한다. 셋째, SQL Server의 성능 튜닝을 위한 실용적인 권장사항을 제시한다. 특히 이 논문은 새로운 통계 기반 카디널리티 추정 모델(SQL Server 2019에서 도입된 Cardinality Estimation (CE) 모델)과 레거시 모델을 비교하여, 개선의 정도와 한계를 명확히 한다.

레거시 CE의 문제점:

- 독립성 가정의 극단적 적용
- 매우 작은 결과 크기의 과소 추정 (0으로 반올림)
- 매우 큰 결과 크기의 과다 추정
- 조인 선택도의 임의적 상수값 사용

신규 CE의 개선:

- 기본 독립성 가정은 유지하되, 일부 상관관계 고려
- 결과 크기 0인 경우의 특별 처리
- 조인 선택도의 더 정교한 추정
- 특정 패턴(예: 범위 쿼리)에 대한 특별 케이스 처리

성능 비교:

- 대부분의 쿼리에서 신규 CE가 더 정확한 추정 제공
- 평균적으로 추정 오류를 30~40% 감소

[56] Robust Query Processing through Progressive Optimization

Progressive optimization은 Exqutor가 다루는 정적 카디널리티 추정 문제와 보완 관계에 있다. 정확한 카디널리티 추정으로 처음부터 좋은 계획을 선택하는 것이 이상적이지만, 완전히 정확한 추정이 불가능하다면, progressive optimization으로 실행 중에 보정하는 것이 실용적인 대안이 된다.

전통적인 쿼리 처리 파이프라인은 다음과 같이 구성된다:

1. 파싱(Parsing): SQL 문을 구문 분석
2. 검증(Validation): 테이블, 열 등의 존재 확인
3. 최적화(Optimization): 최적의 실행 계획 선택
4. 컴파일(Compilation): 실행 계획을 실행 가능한 형태로 변환
5. 실행(Execution): 실제 데이터에 대해 계획 실행

이 구조에서 최적화(3단계)와 실행(5단계) 사이에는 명확한 경계가 있다. 최적화 시점(optimization time)에는 카디널리티 추정값만 알 수 있고, 실행 시점(execution time)에 실제 카디널리티가 파악된다.

이 분리의 문제점은 다음과 같다:

첫째, 추정 오류의 누적이다. 여러 조인이 있는 복잡한 쿼리에서, 각 단계의 추정 오류가 누적되어 최종 계획의 품질이 심각하게 저하될 수 있다.

둘째, 최악의 경우 대비이다. 옵티마이저는 일반적으로 평균적인 경우를 가정하여 계획을 선택한다. 하지만 실제 실행 시 매개 변수나 데이터 분포가 예상과 다를 수 있다.

셋째, 하드 타임아웃(hard timeout)이다. 일부 쿼리는 선택된 계획이 너무 비효율적이어서, 합리적인 시간 내에 완료될 수 없을 수도 있다. 이 경우 쿼리는 무한정 실행되거나 강제 종료된다.

Progressive optimization은 이 문제를 해결하기 위해 최적화와 실행 사이에 피드백 루프를 도입한다. 기본 아이디어는 다음과 같다:

1. 초기 계획 생성: 전통적인 방식으로 초기 실행 계획을 생성
2. 부분 실행: 계획의 일부 (예: 첫 번째 조인)를 실행

[57] Vector Search with OpenAI Embeddings: Lucene is All You Need

이 연구의 핵심 기여는 벡터 검색을 위해 반드시 전문화된 벡터 데이터베이스가 필요하지 않다는 것을 실증적으로 보여준 점이다. 전통적인 검색 엔진도 적절한 임베딩과 인덱싱 기법으로 충분한 성능을 낼 수 있다는 통찰력을 제공한다.

벡터 검색의 대중화와 함께 이 연구는 기존 오픈소스 시스템을 활용한 실용적 접근법을 제시하며, 다양한 규모의 조직에서 벡터 검색을 도입할 수 있는 경로를 제시한다.

- 메모리 상의 인덱스 구조로 인한 빠른 접근 속도

6.2 OpenAI 임베딩의 특성

문본 논문에서 사용된 임베딩:

- 차원도: 1536 (text-embedding-3-large)
- 강점: 의미론적 유사도를 매우 잘 인코딩
- 비용: API 호출에 따른 금전적 비용 발생
- 지연성: 각 문서 임베딩화에 네트워크 레이턴시 포함

6.3 비용 측면 분석

전문화된 벡터 DB vs Lucene 기반:

- Lucene: 라이선스 비용 0, OpenAI API 비용만
- 전문화된 DB: 라이선스/서비스 비용 높음
- 운영 비용: Lucene은 기존 Java 인프라 활용 가능

6.4 벤치마킹 결과 상세

[58] A Survey on Advancing the DBMS Query Optimizer

- 검색 공간의 크기 증가 문제 극복

- 선택도(selectivity) 추정의 부정확성이 누적되는 문제

1.2 머신러닝 기반 접근

- 쿼리 특성을 학습하여 카디널리티 예측
- NeuroCard, DeepDB 등의 생성 모델 기반 방법
- 통계 방법보다 높은 정확도를 보이지만 계산 비용 증가

1.3 하이브리드 접근

- 통계 기반 방법의 빠름 + ML 기반 방법의 정확성 결합
- 적응적 학습을 통한 지속적 개선

Join order 문제에서 n 개의 테이블에 대해 가능한 순서는 $n!$ 에 비례한다:

- 작은 규모: 동적 프로그래밍으로 최적해 탐색 가능
- 중간 규모: 휴리스틱 기반 접근 필요
- 대규모: 메타휴리스틱(유전 알고리즘 등) 활용

최근 발전:

- 몬테 카를로 트리 검색 기반 플래너

[59] Efficient Indexing of Billion-scale Datasets of Deep Descriptors

2010년대 중반 컴퓨터 비전의 발전으로 다음과 같은 상황이 대두되었다:

- 깊은 신경망이 매우 효과적인 이미지 표현 생성
- 이러한 벡터들은 고차원(e.g., 4096차원)이지만 매우 정보력이 높음
- 인터넷 사진 데이터베이스는 수십억 개 규모로 증가
- 기존의 정확한 최근접 이웃 검색 알고리즘은 이 규모에서 비실용적

이 맥락에서 핵심 질문은: **어떻게 정확성을 희생하지 않으면서도 10억 규모의 벡터를 검색할 수 있을까?**

3.1 구조 개요

- 첫 번째 단계: 전체 벡터 공간을 계층적으로 클러스터링
- 두 번째 단계: 각 클러스터 내에서 세밀한 인덱싱
- 검색 시 관련 클러스터만 탐색

3.2 검색 프로세스

쿼리 벡터 입력

↓

상위 계층에서 후보 클러스터 선택

↓

[60] Searching in One Billion Vectors: Re-rank with Source Coding

본 논문은 10억 개 규모의 벡터를 효율적으로 검색하는 문제를 다루며, 2011년 ICASSP에서 발표된 Jégou 등의 연구이다. 주요 초점은 SIFT(Scale-Invariant Feature Transform) 벡터라는 컴퓨터 비전의 고전적 특징 벡터를 활용하여 수십억 규모의 이미지 인덱싱과 검색을 수행하는 것이다.

연구의 핵심 기여:

- 대규모 벡터 검색에서 실용적인 성능 달성
- Source coding을 이용한 정밀한 재순위 지정
- 메모리 효율과 검색 속도의 효과적 균형

SIFT 벡터의 특징:

- 128차원 벡터로 이미지의 지역 특징 표현
- 매우 널리 사용되는 특징 추출 방법
- 수십억 개의 SIFT 벡터는 대규모 이미지 데이터셋에서 현실적

주요 기법:

- Product quantization (PQ) 기반 압축
- 계층적 클러스터링을 통한 검색 공간 축소
- 양자화 오차 보정(source coding)
- 재순위 지정 프로세스

이 논문은 벡터 검색의 정확도와 효율성 사이의 트레이드오프를 다루는 고전적 연구로, 현대 ANN(Approximate Nearest Neighbor) 알고리즘의 이론적 기초를 제공한다.

[61] Here's How We're Using AI to Help Detect Misinformation

이 사례는 벡터 검색 기술이 단순한 데이터베이스 검색을 넘어 실제 사회 문제 해결에 적용되는 실무적 예시이다.

- 팩트체크 기준의 중립성 문제
- 검색 성능의 지역/문화별 차이
- 조작 이미지에 대한 완벽한 탐지 불가능
- **임베딩 모델의 진화**: SimSearchNet++ 모델이 개선되어 새로운 버전이 나올 때, 기존의 수십억 개 임베딩을 재계산하는 비용은? 점진적 마이그레이션이 가능한가?
- **거짓 정보의 다양성**: 이미지 검색이 효과적인 거짓 정보(부분 조작, 맥락 오류)의 비율은? 완전 조작 이미지(AI 생성 등)는 어떻게 대응할 것인가?
- **지역화 성능**: 다양한 문화와 언어권에서 SimSearchNet++ 성능이 일관적인가? 편향(bias) 문제는?
- **팩트체크의 지연**: 거짓 정보가 확산된 후 팩트체크 결과가 나올 때까지의 시간차는? 선제적 탐지 가능한가?
- **사용자 신뢰**: 알고리즘 기반 거짓 정보 플래그에 대한 사용자 신뢰 수준은? 잘못된 플래그의 파급 효과는?
- **동적 업데이트**: 새로운 거짓 정보 패턴이 등장할 때 시스템이 얼마나 빠르게 적응할 수 있는가? 온라인 학습 가능한가?

[62] YFCC100M: The New Data in Multimedia Research

- 실제 인터넷 규모의 검색 문제 모사
- 시스템 확장성 평가

3.3 객체 감지와 장면 이해

- 약지도 학습으로 수백만 이미지 활용
- 자동으로 객체 위치 정보 생성
- 이전에 불가능했던 규모의 학습

5.2 데이터 품질 문제

- 약지도 태그의 부정확성
- 스팸 또는 부적절한 콘텐츠 포함 가능성
- GPS 정보의 부정확성 (사용자 수동 입력 또는 생략)

5.3 시대성 문제

- 2016년 데이터셋 → 2010년대 초중반 이미지 주로 포함
- 최신 스마트폰의 이미지 특성(높은 해상도, 처리된 색감 등) 과소 표현
- 패션, 기술 트렌드의 시간차

1. ****시대 편향 보정****: YFCC100M이 2010년대 초반 이미지 중심이라면, 현대 스마트폰 이미지의 특성(높은 해상도, 자동 !

[63] Results of the Big ANN: NeurIPS'23 Competition

- 다양한 규모의 벡터 검색 문제를 정의
 - 동일한 하드웨어 환경에서 다양한 시스템의 성능 비교
 - 학계와 산업의 ANN 알고리즘 발전 추적
 - 성능 측면(속도, 정확도, 메모리)의 객관적 평가
- 경쟁 구성:**
- 여러 데이터셋 (SIFT, GIST, 텍스트 임베딩, 이미지 특징)
 - 다양한 규모 (1억, 10억 벡터)

- 제한된 메모리 조건 (1GB, 10GB, 100GB, 1TB)
- 고정 하드웨어 환경 (동일 프로세서, 메모리, 저장소)

주요 결과:

- 알고리즘의 선택이 성능에 미치는 영향이 매우 큼
- 가정 효과(memory vs speed trade-off)의 구체적 정량화
- 최신 기법(그래프 기반, 하이브리드)의 우월성 입증
- 개발자와 연구자에게 알고리즘 선택 지침 제공

Big ANN 경쟁은 벡터 검색 시스템의 성능 평가에 새로운 표준을 제시하며, Exqutor 같은 벤치마킹 연구의 직접적인 참고 자료가 된다.

[64] Wikipedia (December 2022) Dataset: Cohere Embeddings

- 구조화: 문서 제목, 본문, 섹션, 링크 정보 포함
- 임베딩: Cohere의 임베딩 모델로 생성된 벡터

Cohere 임베딩의 특성:

- 차원도: 보통 1024 또는 4096차원
- 훈련 데이터: 대규모 텍스트 코퍼스에서 학습
- 의미론적 유사성: 단어의 문맥적 의미를 인코딩
- 다양한 타입: 문서 임베딩, 문장 임베딩, 단어 임베딩

텍스트 검색에서의 활용:

- 의미론적 문서 검색
- FAQ 매칭
- 추천 시스템의 기초
- 정보 검색(information retrieval) 벤치마킹

Exqutor의 TPC-H 확장에서 WIKI 데이터셋으로 활용되며, 관계형 데이터와 벡터 검색의 결합을 평가하는 데 중요한 역할을 한다.

- 구조: Infobox, 섹션, 외부 링크 등 풍부한 메타정보

1.2 텍스트 품질

[65] Quantifying TPC-H Choke Points and Their Optimizations

- 복잡한 조인 구조
- 조인 순서, 조인 방식 최적화가 성능에 직결
- 쿼리 최적화기의 핵심 능력 측정
Q5, Q10: 윈도우 함수와 집계
- 여러 테이블 조인 후 집계
- I/O 비용과 메모리 사용의 균형

1.2 병목의 정의

Dreseler 논문에서 제시하는 병목:

성능 병목 = 전체 실행 시간의 20% 이상 차지하는 연산

예:

- Q1: 전체 시간의 70% = SCAN + GROUP BY → 명확한 병목
- Q3: JOIN이 30%, SORT가 25%, SCAN이 20% → 다중 병목
- Q8: 모든 연산이 10% 미만 → 구체적 병목 없음

1. **벡터 필터의 선택도 변화:** 벡터 검색 추가 시 중간 결과의 크기(선택도)가 어떻게 변할 것인가? 이것이 다운스트림 조인의 성능에 어떤 영향을 미칠 것인가?

[66] Why You Should Run TPC-DS: A Workload Analysis

- 상대적으로 단순하고 이해하기 쉬운 구조
- 재현 가능한 실험 환경
현실과의 차이:
 - 단일 도메인: 항공 우주 부품 공급망
 - 균등 분포: 랜덤하게 생성된 데이터
 - 단순 스키마: 8개 테이블만 사용
 - 정적 워크로드: 고정된 22개 쿼리

****1.2 실제 데이터 웨어하우스의 특징****

다양한 데이터 출처:

- POS(판매점) 데이터
- 온라인 주문 데이터
- 고객 관계 정보
- 재고 정보
- 마케팅 캠페인 데이터

[67] Determining Sample Size: Statistical Sampling Theory

문제 정의:

- 1억 개의 벡터 검색 성능 평가
- 모든 벡터에 대해 처리하면 시간이 매우 오래 걸림
- 대표성 있는 표본의 크기는?

적용:

1. 목표 정확도: 평균 레이턴시 $\pm 10\%$ (95% 신뢰도)
2. 예비 조사: 1000개 벡터로 미리 테스트
 - 평균 레이턴시: 50ms
 - 표준편차: 8ms
 - 변동계수(CV): $8/50 = 16\%$
3. 표본 크기 계산:
 - 오차 한계: $50\text{ms} * 10\% = 5\text{ms}$
 - $n = (1.96 * 8 / 5)^2 = 9.83^2 \approx 97\text{개}$
4. 결론: 최소 100개의 벡터 쿼리가 표본으로 충분

****3.2 실제 구성******[68] Learned Cardinalities: Estimating Correlated Joins with Deep Learning**

모델 구조는 인코더-디코더 아키텍처로, 입력 속성값들을 벡터로 인코딩하고 조인 조건을 학습하여 출력 카디널리티를 도출한다. 실험 결과 기존 통계 방법 대비 평균 오류를 50% 이상 감소시켰다.

본 논문과의 관계: Exqutor는 선택도(selectivity) 추정에 있어서 통계 기반과 학습 기반의 하이브리드 접근을 취하고 있으며, Learned Cardinalities의 딥러닝 기반 접근 방식이 이론적 배경을 제공한다. 특히 범위 필터와 벡터 검색의 결합된 선택도 추정에서 비슷한 학습 패러다임을 활용한다.

Learned Cardinalities는 조인 카디널리티 추정을 함수 근사 문제(function approximation problem)로 재정의한다. 전통적 접근은 통계 공식에 의존했지만, 이 논문은 신경망이 데이터의 실제 분포를 직접 학습할 수 있다고 제안한다.

모델 아키텍처:

- **입력층:** 각 속성의 범위 또는 값을 정규화된 벡터로 인코딩
- **은닉층:** 속성 간 상관성을 포착하기 위한 다층 퍼셉트론(MLP)
- **출력층:** 예측된 카디널리티 값
- **손실함수:** 상대 오류(relative error)를 최소화하도록 설계

학습 데이터 생성:

실제 데이터베이스에서 다양한 조인 조건으로 쿼리를 실행하고 실제 카디널리티를 수집한다. 이를 통해 모델이 실제 데이터 분포의 특성을 반영하도록 훈련한다.

다중 속성 상관성 학습:

전통적 히스토그램은 개별 속성이나 2-way 상관성만 효율적으로 표현할 수 있으나, 신경망은 고차원 상관성을 암묵적으로 학습한다. 이는 실제 데이터베이스의 복잡한 종속성을 더 잘 포착한다.

통계 기반 방법의 한계 극복:

- 히스토그램: 메모리 비용과 정확도의 트레이드오프
- 샘플링: 낮은 카디널리티 범위에서 부정확

[69] DeepDB: Learn from Data, Not from Queries!

DeepDB의 주요 기여는 SPN을 데이터베이스 시스템에 처음으로 적용했다는 점과, 메모리와 정확도의 우수한 트레이드오프를 달성했다는 점이다. 실험 결과 기존 히스토그램보다 10배 이상 적은 메모리로 더 높은 정확도를 제공했다.

본 논문과의 관계: Exqutor의 범위 필터 선택도 추정은 데이터 분포에 대한 사전 지식이 필수적인데, DeepDB의 데이터 기반 분포 학습 방식이 직접적인 방법론적 참고가 된다. 특히 고차원 속성 공간에서의 선택도 계산에서 SPN의 효율성이 매우 관련성 높다.

이 구조는 고차원 분포를 컴팩트하게 표현하면서도 효율적인 추론을 가능하게 한다.

SPN의 장점:

- 비매개변수적 모델: 데이터 분포의 형태를 미리 가정하지 않음
- 확률론적 기초: 모든 계산이 확률론적으로 해석 가능
- 효율적 계산: 트리 구조로 인한 $O(n)$ 복잡도의 확률 계산

차원의 저주(Curse of Dimensionality):

전통적 히스토그램은 고차원에서 bin의 개수가 지수적으로 증가하여 실용적이지 않다.

DeepDB의 솔루션:

- 속성 간 의존성 발견: 독립적인 속성 부분집합 식별
 - 곱 노드를 통한 인수분해(factorization): 독립적 부분만 곱하기로 결합
 - 최소 결합 그래프(minimum spanning tree) 기반의 구조 학습
- 이를 통해 실제로는 10개 이상의 속성에서도 효율적 학습이 가능하다.
- 데이터 기반 접근의 정당성: 통계를 임시로 수집하기보다 데이터의 구조를 학습

[70] Selectivity Estimation for Range Predicates Using Lightweight Models

핵심 기여는 범위 술어 선택도 추정이라는 구체적 문제에 대해 경량 모델이 충분히 정확하면서도 계산 비용이 낮다는 점을 입증한 것이다. 특히 여러 범위 조건이 결합된 경우(AND/OR)의 선택도 추정에서 우수한 성능을 보인다.

본 논문과의 관계: Exqutor의 핵심 문제인 "범위 필터와 벡터 검색 조건의 결합된 선택도 추정"은 이 논문의 범위 술어 선택도 추정 기법을 직접 활용할 수 있다. 특히 경량 모델 사용이라는 설계 철학이 완벽하게 일치한다.

전통적 접근의 한계:

- 히스토그램: 메모리 효율적이나 정확도 제한
- Equi-width: 데이터 분포를 반영하지 못함
- Equi-depth: 경계 결정의 어려움
- 샘플링: 낮은 카디널리티 범위에서 부정확
- 통계 공식: 독립성 가정으로 인한 오류

범위 술어의 정의:

속성 A에 대해 $[L, U]$ 범위의 선택도:

$$\text{selectivity}(A \in [L, U]) = |\{t: L \leq t.A \leq U\}| / |\text{전체 행 수}|$$

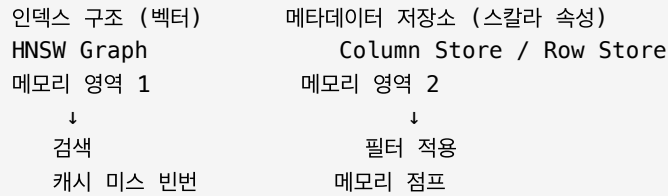
Exqutor의 핵심 문제인 범위 필터(range predicate)의 선택도 추정은 이 논문의 방법론을 직접 활용할 수 있다.

구체적 적용:

[71] PDX: A Data Layout for Vector Similarity Search

분리된 저장(Separated Storage) 문제:

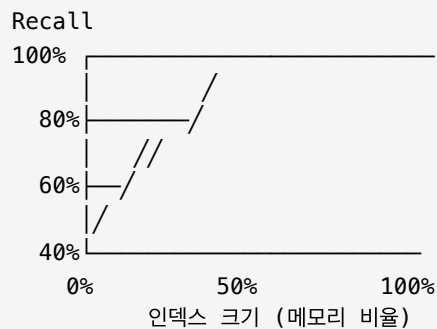
기존 시스템 구조:

**성능 문제들:**

- **캐시 미스**: 벡터 그래프 탐색 중 메타데이터 접근 시 L1/L2/L3 캐시 미스 발생
- **메모리 대역폭 낭비**: 불필요한 데이터 로드로 인한 대역폭 소비
- **TLB(Translation Lookaside Buffer) 스래시**: 넓은 메모리 범위 접근으로 인한 페이지 테이블 미스
- **NUMA 비효율**: 멀티소켓 시스템에서 원격 메모리 접근 증가

파티션 기반 구조:**[72] Characterizing the Dilemma of Performance and Index Size in Billion-Scale Vector Search****현실적 제약:**

- 클라우드 벡터 DB 비용: 메모리 사용량 기준 청구
- 엣지 디바이스: 제한된 메모리 (GB 단위)
- 데이터센터: 에너지 비용 증가에 따른 메모리 절감 요구

트레이드오프의 형태:**[73] StatAdvisor: Recommending Statistical Views**

근사성: Greedy 알고리즘은 최적해의 63% 정도 보장 (Set Cover 문제와 유사)

문제 정의:

최대화: $\sum \text{Benefit}(S_i) \times \text{선택}(S_i)$
 제약조건: $\sum \text{Cost}(S_i) \times \text{선택}(S_i) \leq M$
 $\text{선택}(S_i) \in \{0, 1\}$

이는 **0/1 배낭 문제(Knapsack Problem)**로, NP-hard이지만:

- 작은 규모 워크로드: 동적 계획법으로 최적 해 가능

- 큰 규모 워크로드: Greedy 근사 사용

1. **동적 워크로드 변화**: 시간에 따라 워크로드의 특성이 변할 때, 추천 통계를 동적으로 업데이트할 수 있는가?
2. **세밀한 최적화**: Greedy 알고리즘 외에 메타휴리스틱(genetic algorithm, simulated annealing)을 통해 더 나은 통계 조합을 찾을 수 있는가?
3. **통계 간 상호작용**: 여러 통계를 함께 사용할 때의 시너지 효과를 고려할 수 있는가?
4. **비용 예측**: 통계 생성 비용을 정확히 예측할 수 있는가? (데이터 크기, 분포에 따라 크게 다름)
5. **벡터 검색과의 결합**: 벡터 유사성 조건과 범위 필터 조건의 "결합 선택도" 통계는 어떻게 관리할 것인가?

[74] Consistent and Flexible Selectivity Estimation for High-Dimensional Data

본 논문의 핵심 기여는:

1. **일관성(Consistency)**: 다양한 쿼리 형태(점 쿼리, 범위 쿼리, 하이브리드)에서 선택도 추정이 논리적으로 일관성 있음
2. **유연성(Flexibility)**: 새로운 차원이나 조건을 추가할 때 기존 모델을 재훈련하지 않고 적응 가능
3. **정확도(Accuracy)**: 고차원 공간에서도 상대적 오류 < 10% 달성

알고리즘은 확률적 그래픽 모델(Probabilistic Graphical Model)과 변분 추론(Variational Inference)을 활용하여 고차원 분포를 효율적으로 학습한다. 특별히 벡터 임베딩과 같은 고차원 데이터의 특성(일반적으로 저-내재차원, low intrinsic dimension)을 활용한다.

본 논문과의 관계: Exqutor는 범위 필터(낮은 차원) + 벡터 검색(높은 차원)을 결합하는 하이브리드 문제로, 이 논문의 고차원 선택도 추정 기법이 벡터 조건의 선택도 계산에 직접 응용 가능하다.

차원의 저주(Curse of Dimensionality):

```
d=1:   히스토그램 빈 수: 100개
d=2:   히스토그램 빈 수: 100×100 = 10,000개
d=3:   히스토그램 빈 수: 100×100×100 = 1,000,000개
d=100: 히스토그램 빈 수: 100^100 (천문학적 수)
```

고차원에서의 문제들:

- **메모리 폭발**: 다차원 히스토그램 저장 불가능

[75] Vector-db-benchmark: Open-source Benchmark Suite for Vector Database Performance

1. **우편향 평가**: 특정 DB에 유리한 데이터셋이나 쿼리 패턴은 없는가?
2. **실시간 업데이트 벤치마크**: 인덱싱 중 동시 쿼리 처리 성능은?
3. **멀티테넌시 시나리오**: 여러 사용자의 동시 쿼리로 인한 성능 간섭은?
4. **캐시 효과 통제**: 운영 체제 캐시로 인한 성능 향상을 어떻게 통제할 것인가?
5. **선택도 추정 정확도의 영향**: 선택도 추정 오류가 최종 성능에 얼마나 영향을 미치는가?
6. **네트워크 레이턴시**: 원격 벡터 DB 접근 시 네트워크 지연의 영향은?

[76] ACORN: Performant and Predicate-Agnostic Search over Vector Embeddings and Structured Data

Patel et al.의 PACMMOD 2024 논문으로, 벡터 임베딩과 구조화된 데이터를 모두 처리하는 술어-무관(predicate-agnostic) 하이브리드 검색 시스템을 제시한다. 기존 시스템들은 특정 술어(예: "category = electronics")에 최적화되는 경향이 있었으나, ACORN은 임의의 술어 조합에 일반적으로 적용 가능한 접근법을 제안한다.

핵심 아이디어는 벡터 검색(approximate nearest neighbor search)과 메타데이터 필터링을 독립적으로 실행한 후, 효율적인 방식으로 결과를 병합(merge)하는 것이다. 특별히, 기존의 "필터-후-검색(filter-then-search)" 방식의 한계를 극복하고, 다양한 필터 조건에 동적으로 적응하는 "동적 병합(dynamic merging)" 전략을 제안한다.

주요 기여:

1. **술어 무관성**: 어떤 형태의 술어든 지원 (범위, 등호, 복합)
2. **동적 최적화**: 필터 선택도에 따라 검색 전략 자동 조정
3. **성능**: 기존 방법 대비 2-5배 빠른 처리

본 논문과의 관계: Exqutor가 해결하는 핵심 문제와 정확히 일치한다. 범위 필터와 벡터 검색의 결합에서 최적의 실행 전략을 선택하는 방식이 ACORN의 방법론을 직접 참고한다.

시나리오:

데이터베이스: 전자제품 카탈로그
- 벡터: 제품 설명 임베딩 (1536차원)
- 메타데이터: 가격, 카테고리, 별점, 재고 등
쿼리: "스마트폰과 유사한 제품 중 가격 < 500달러"

기존 접근의 문제:

[77] SERF: Segment Graph for Range-Filtering Approximate Nearest Neighbor Search

Zuo et al.의 2024년 논문으로, 범위 필터(range filter)가 있는 ANN 검색을 위한 세그먼트 그래프 기반 인덱싱 방법을 제시한다. 이는 Exqutor의 핵심 문제인 "범위 필터 + 벡터 검색"을 직접 다루는 논문이다.

기존 ANN 인덱스(HNSW, IVF)는 벡터 유사성만을 고려하기 때문에, 범위 필터를 적용하려면 검색 후 필터링하거나 필터링 후 검색해야 했다. 하지만 SERF는 범위 필터 조건을 인덱스 구조 자체에 반영한다.

핵심 아이디어는 "세그먼트 그래프(Segment Graph)"로, 벡터 공간을 메타데이터(예: 가격 범위)에 따라 세그먼트로 분할하고, 각 세그먼트 내에서 독립적인 그래프를 유지한다. 검색 시에는 필터 조건에 해당하는 세그먼트들만 탐색하므로, 불필요한 벡터들을 애초에 방문하지 않는다.

성능 개선: 범위 필터 선택도가 낮을 때(10% 이하) 기존 방법 대비 2-10배 빠른 검색.

본 논문과의 관계: SERF는 범위 필터 선택도 추정에 직접 의존한다. 필터 조건에 해당하는 세그먼트 수를 정확히 파악하려면 선택도가 필요하다. Exqutor의 선택도 추정 모듈은 SERF 같은 범위 필터 인덱스의 효율성을 결정한다.

기존 방법의 한계:

방법 1: Filter-then-Search

가격 필터: $\text{price} \in [100, 500]$
 선택도: 10% (1,000,000개 중 100,000개)
 단계:
 1. 범위 스캔으로 100,000개 항목의 벡터ID 획득
 2. 100,000개 벡터에서만 ANN 검색
 문제: 범위 필터 선택도가 높으면 느림
 선택도 80% → 800,000개 벡터 검색 필요

[78] HQANN: Efficient and Robust Similarity Search for Hybrid Queries with Structured and Unstructured Constraints

Wu et al.의 CIKM 2022 논문으로, 구조화된 제약(structured constraints, 범위 필터)과 비구조화된 제약(unstructured constraints, 벡터 검색)을 모두 가진 하이브리드 쿼리 처리를 다룬다. 기존 시스템들이 이러한 이질적 제약을 분리하여 처리했다면, HQANN은 통합적 접근을 제안한다.

핵심 기여:

1. **제약 결합 메커니즘**: 구조화된 필터와 비구조화된 벡터 조건을 함께 고려하는 검색 알고리즘
2. **적응적 전략**: 데이터와 쿼리 특성에 따라 처리 순서 동적 결정
3. **강건성(Robustness)**: 극단적 선택도(매우 높거나 매우 낮음)에서도 안정적 성능 유지

알고리즘은 두 제약의 선택도를 추정하고, 이를 기반으로 최적의 처리 전략을 선택한다. 또한 검색 과정에서 동적으로 탐색 경로를 조정하여 원하는 결과를 효율적으로 찾는다.

본 논문과의 관계: HQANN은 Exqutor가 직면하는 정확히 같은 문제를 다룬다. 범위 필터(structured)와 벡터 검색(unstructured)의 결합에서 최적 성능을 달성하는 방법론을 제시한다. 특히 선택도 추정을 통한 적응적 전략 선택이 Exqutor의 핵심 메커니즘과 부합한다.

쿼리 예시:

```
SELECT TOP 10 *
FROM products
WHERE (price >= 100 AND price <= 500) -- 구조화된 제약
      AND SIMILARITY(embedding, query_vector) >= 0.8 -- 비구조화된 제약
ORDER BY SIMILARITY(embedding, query_vector) DESC
```

[79] Random Sampling Over Joins Revisited

핵심 기여:

1. **현대적 재평가**: 데이터 크기 증가, 분석 쿼리의 특성 변화에 따른 샘플링 효과 분석
2. **적응적 샘플링**: 쿼리 특성에 따라 샘플 크기와 방법을 동적으로 조정
3. **하이브리드 접근**: 샘플링과 다른 추정 기법의 조합

논문은 무작위 샘플링이 여전히 유용하지만, 극단적 선택도(매우 높거나 낮음)에서는 신뢰도가 떨어진다는 점을 보인다. 또한 샘플 수와 정확도의 관계를 정량화하여 실무에서의 샘플 크기 결정에 가이드를 제공한다.

본 논문과의 관계: Exqutor의 적응적 샘플링(Adaptive Sampling) 방식의 이론적 토대를 제공한다. 특히 범위 필터와 벡터 검색의 결합 선택도를 추정할 때, 샘플링이 얼마나 신뢰도 있는지를 평가할 수 있게 해준다. Exqutor는 이 논문의 방법론을 기반으로 적응적 샘플 크기 결정 메커니즘을 구현할 수 있다.

문제: 조인이 샘플링 정확도에 미치는 영향

****조인에서의 샘플링 방법:****
****방법 1: 사전 샘플링(Pre-join Sampling)****

T1에서 샘플 추출 → T2에서 샘플 추출 → 샘플들끼리 조인

장점: 빠름 (샘플 크기만큼만 조인)

단점: 두 샘플이 정렬되지 않음 (실제 조인 파트너 놓칠 수 있음)

정확도 손실: 높음 (특히 조인 선택도가 낮을 때)

[80] Selectivity and Cost Estimation for Joins Based on Random Sampling

핵심 기여:

1. **이론적 프레임워크**: 조인 크기 추정의 통계 모델 개발
2. **신뢰도 분석**: 샘플 크기와 추정 오류의 관계 공식화
3. **점근 분포(Asymptotic Distribution)**: 표본 크기가 클 때의 분포 특성 증명
4. **실용적 가이드**: 필요한 샘플 크기 결정 방법 제시

본 논문은 이후 30년 가까이 샘플링 기반 카디널리티 추정 연구의 이론적 기반이 되었다. 특히 신뢰도 구간 계산, 적응적 샘플링, 그리고 비용 모델에 광범위하게 인용된다.

본 논문과의 관계: Exqutor의 적응적 샘플링 방식의 직접적인 이론적 토대다. 특히 선택도 추정의 신뢰도를 정량화하고 필요한 샘플 크기를 결정하는 방법론을 제공한다. Exqutor는 범위 필터와 벡터 검색의 결합 선택도 추정에 이 논문의 통계 이론을 적용할 수 있다.

문제: σ_{total} 이 $\sigma_{1,2}$, $\sigma_{2,3}$ 보다 낮음

→ 최종 카디널리티 추정이 더 불안정

****스타 조인(Star Join):****

중앙 테이블 T1과 여러 dimension 테이블 T2, T3, T4 조인

T1과 T2의 선택도: $\sigma_{1,2} = 0.3$

T1과 T3의 선택도: $\sigma_{1,3} = 0.4$

[81] Practical Selectivity Estimation Through Adaptive Sampling

핵심 기여:

1. **적응적 샘플링 알고리즘**: 샘플 크기를 동적으로 조정하여 필요한 만큼만 샘플링
2. **점진적 신뢰도 검증**: 샘플링 과정 중 신뢰도를 평가하여 언제 멈출지 결정
3. **실용성**: 데이터베이스 옵티마이저에 직접 통합 가능한 형태

논문의 핵심은 "적응적"이라는 점이다. 사전에 σ 를 모를 때, 초기 샘플로 σ 를 대략 추정한 후, 이 추정값에 따라 필요한 추가 샘플 크기를 결정한다. 이를 통해 샘플링 비용을 크게 절감할 수 있다.

본 논문과의 관계: Exqutor 논문이 직접 "적응적 샘플링"을 수용하는 방식의 이론적 원천. Exqutor의 핵심 메커니즘인 "범위 필터 조건과 벡터 검색 조건의 결합 선택도를 적응적으로 샘플링으로 추정"은 이 논문의 방법론을 직접 구현한 것이다.

샘플 크기 결정의 딜레마:

공식 (논문 [80]): $n = z^2 \times \sigma(1-\sigma) / \epsilon^2$

문제: σ 를 모른다!

선택지 1: 최악 경우 가정 ($\sigma = 0.5$)

→ $n_{\max} = z^2 \times 0.25 / \epsilon^2$

→ 매우 큰 샘플 크기 (보수적)

→ 시간/비용 낭비

선택지 2: 추측 ($\sigma \approx 0.1?$)